

INF1161

Réseaux mobiles et communications sans fil

UNIVERSITÉ TÉLUQ

Devoir 2

Simulation du protocole CSMA/CA à événements discrets

Date de remise : 20-05-2026

Présenté à : Es-Said Sabir

Étudiante : Melissa Moya

Numéro d'étudiant : 24332062

Table des matières

1. Introduction.....	3
2. Méthodologie	4
3. Architecture générale du simulateur.....	5
3.1 Moteur de simulation à événements discrets.....	5
3.2 Modèle des stations.....	6
3.3 Implémentation du protocole CSMA/CA.....	6
3.4 Calcul des métriques de performance	8
4. Choix techniques	9
5. Résultats expérimentaux	10
5.1 Performance sous charge modérée ($\lambda = 20$ paquets/s par station)	10
5.2 Performance sous forte charge ($\lambda = 200$ paquets/s par station).....	12
5.3 Impact de la taille minimale de la fenêtre de contention (W_{min})	13
5.4 Impact du nombre maximal de retransmissions (K_{max})	14
5.5 Impact du mécanisme RTS/CTS	15
6. Analyse et interprétation des résultats.....	16
7. Conclusion.....	17
Bibliographie	18
Annexe	19
Annexe A –Présentation du dépôt GitHub.....	19
Annexe B – Commandes de simulation et génération des graphiques.....	20
Annexe C - Code source du simulateur intégralement commenté.....	22

1. Introduction

Les réseaux sans fil reposent sur des mécanismes efficaces de partage du médium afin de permettre à plusieurs stations de communiquer sur un même canal. Parmi ces mécanismes, le protocole CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance), soit l'accès multiple par détection de porteuse avec évitement des collisions, joue un rôle central dans les réseaux de type IEEE 802.11, soit les réseaux Wi-Fi, en assurant une gestion équitable de l'accès au canal et en limitant les collisions. Il constitue la base de la fonction DCF (Distributed Coordination Function), principale méthode d'accès au médium définie par le standard IEEE 802.11.

Dans ce contexte, ce projet propose la conception et la mise en œuvre d'un simulateur à événements discrets du protocole CSMA/CA, visant à modéliser le fonctionnement de la couche MAC dans un environnement sans fil. Afin de simplifier le modèle, l'écoute du canal est réalisée de manière abstraite en supposant que le simulateur dispose d'une vue globale des états de toutes les stations, ce qui évite de modéliser la propagation physique du signal. Le canal est partagé selon le mécanisme d'attente binaire exponentielle défini par IEEE 802.11, et les collisions ne sont détectées qu'après la tentative de transmission.

L'objectif est de reproduire le comportement de plusieurs stations en compétition pour l'accès au canal et d'évaluer les performances du protocole à l'aide de différentes métriques telles que le débit, le taux de collision et le délai moyen de transmission. Les arrivées de paquets sont modélisées par des intervalles exponentiels. Toutefois, conformément aux hypothèses du modèle, une station ne génère pas de nouveau paquet tant que le paquet courant n'a pas été transmis avec succès ou abandonné après dépassement du nombre maximal de tentatives. Le processus global correspond ainsi à un processus de renouvellement et non à un processus de Poisson strict. Le comportement de chaque station est contrôlé par les paramètres $W_{\min}$, $W_{\max}$ et $K_{\max}$, ainsi que par les temporisations DIFS, SIFS et slot time, conformément au standard IEEE 802.11b. Notons que $K_{\max}$ définit le nombre maximal de retransmissions, soit $K_{\max} + 1$ tentatives de transmission au total avant abandon du paquet (incluant la première tentative).

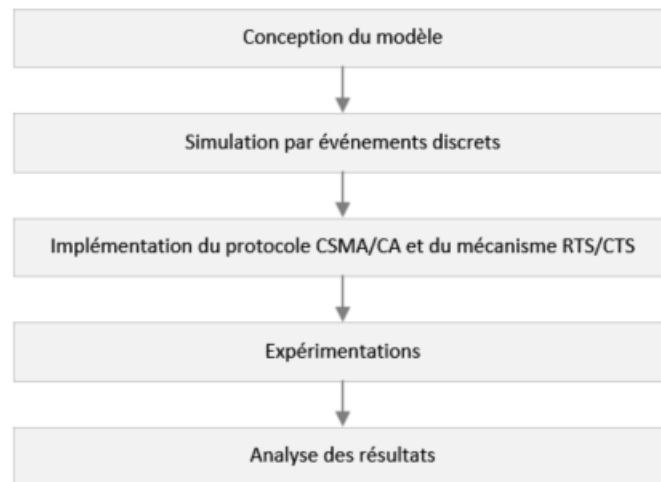
Une extension intégrant le mécanisme RTS/CTS ainsi que la réservation du médium par le vecteur d'allocation réseau (NAV) est également étudiée afin d'analyser son impact sur le comportement global du système. Le simulateur permet ainsi d'évaluer l'effet de différents paramètres du protocole, notamment le nombre de stations, la taille de la fenêtre de contention et le nombre maximal de retransmissions.

La suite du rapport présente la méthodologie de conception, l'architecture du simulateur et les choix techniques retenus, puis les résultats expérimentaux et leur analyse.

2. Méthodologie

Le développement du simulateur repose sur une approche progressive visant à modéliser fidèlement le protocole CSMA/CA tout en conservant une structure simple et modulable. La méthodologie adoptée décompose le système en cinq étapes clairement définies, comme illustré à la Figure 1.

Figure 1 – Étapes de conception du simulateur CSMA/CA



Dans un premier temps, le choix d'un modèle à événements discrets a été retenu afin de représenter l'évolution du système dans le temps. Cette approche permet de simuler efficacement les interactions entre les stations en ne traitant que les événements significatifs, soit les arrivées de paquets, les tentatives de transmission, les mises à jour du backoff et les fins de transmission. Le temps est géré de manière discrète et les événements sont planifiés en fonction du slot time du protocole, ce qui garantit un traitement chronologique cohérent.

Dans un second temps, un modèle de station a été défini afin de représenter le comportement individuel des entités du réseau. Chaque station maintient un état interne comprenant le paquet en cours de traitement, le compteur de backoff, le nombre de tentatives de retransmission ainsi que les temporisations associées au protocole. L'écoute du canal est modélisée de manière abstraite en supposant que le simulateur dispose d'une vue globale des états de toutes les stations, ce qui permet de représenter le mécanisme de détection de porteuse sans recourir à un modèle physique détaillé. Conformément aux hypothèses du sujet, chaque station ne traite qu'un seul paquet à la fois.

Dans un troisième temps, la logique du protocole CSMA/CA a été implémentée, incluant le mécanisme d'attente binaire exponentielle, la gestion des collisions et la retransmission des paquets en cas d'échec. Une extension intégrant le mécanisme RTS/CTS et la réservation du

médium par le vecteur d'allocation réseau (NAV) a également été ajoutée afin de permettre une comparaison entre différentes stratégies d'accès au médium. Le fonctionnement détaillé de ces mécanismes est présenté à la section 3.3.

Par la suite, une série d'expérimentations a été mise en place en faisant varier les paramètres du système, notamment le nombre de stations, le taux d'arrivée des paquets ainsi que les paramètres du protocole tels que $W_{\min}$, $W_{\max}$, $K_{\max}$ et les temporisations DIFS, SIFS et slot time. Afin d'assurer la fiabilité des résultats, chaque configuration est simulée plusieurs fois et les résultats sont moyennés, réduisant ainsi l'impact des fluctuations aléatoires propres au modèle.

Enfin, les performances du système sont évaluées à l'aide de trois métriques principales, soit le débit moyen, le taux de collision et le délai moyen de transmission. Ces indicateurs permettent d'analyser le comportement du protocole et d'identifier ses limites en fonction des conditions de charge du réseau.

3. Architecture générale du simulateur

Le simulateur développé repose sur une architecture modulaire permettant de représenter de manière structurée le fonctionnement du protocole CSMA/CA. Cette organisation facilite à la fois l'implémentation du modèle et l'analyse des interactions entre les différents éléments du système. L'ensemble s'articule autour de trois composants principaux, soit un moteur à événements discrets, un modèle de stations et un module d'implémentation du protocole et de calcul des métriques.

3.1 Moteur de simulation à événements discrets

Le cœur du simulateur est un moteur à événements discrets permettant de modéliser l'évolution temporelle du système. Dans cette approche, le temps progresse de manière discontinue en fonction des événements significatifs, soit les arrivées de paquets, les événements d'incrément de slot (`slot_tick`), les fins de transmission de données et, en mode RTS/CTS, les fins d'émission de trames RTS.

Les événements sont gérés à l'aide d'une file de priorité à tas minimal, implémentée avec la bibliothèque `heapq` de Python, ce qui garantit leur traitement dans l'ordre chronologique. Chaque événement est associé à un numéro de séquence monotone qui permet de départager de manière déterministe les événements partageant le même instant, assurant ainsi la reproductibilité des simulations. Cette approche évite une simulation à pas de temps constant et améliore significativement l'efficacité du modèle.

Par ailleurs, un mécanisme de jeton permet d'invalidiser les événements d'incrément de slot devenus obsolètes sans les retirer physiquement de la file, ce qui simplifie la gestion des

transitions d'état du canal. Cette structure favorise également la flexibilité du simulateur, puisqu'elle permet d'intégrer facilement de nouveaux types d'événements. Le découplage entre la gestion temporelle et la logique des stations facilite ainsi l'évolution du modèle et sa réutilisation dans différents scénarios.

3.2 Modèle des stations

Le simulateur considère un ensemble de stations indépendantes partageant un même canal de communication. Chaque station est modélisée par un état interne évolutif comprenant le paquet actif en cours de traitement, le compteur de backoff courant, le nombre de tentatives de retransmission effectuées ainsi qu'un champ de réservation indiquant jusqu'à quel instant le médium est réservé par le vecteur d'allocation réseau (NAV) en mode RTS/CTS.

Conformément aux hypothèses du modèle, une station ne traite qu'un seul paquet à la fois. Aucune nouvelle arrivée n'est planifiée tant que le paquet courant n'a pas été transmis avec succès ou abandonné après dépassement de $K_{\max}$ tentatives. En cas de succès, la fenêtre de contention est réinitialisée à $W_{\min}$ et un nouveau tirage de backoff est effectué pour le paquet suivant.

Le comportement des stations est régi par les paramètres du standard IEEE 802.11b, notamment les temporisations DIFS, SIFS et slot time. Ce modèle permet de représenter de manière cohérente le comportement individuel de chaque station ainsi que ses interactions avec le canal partagé et les autres stations en compétition dans un contexte de contention distribuée.

3.3 Implémentation du protocole CSMA/CA

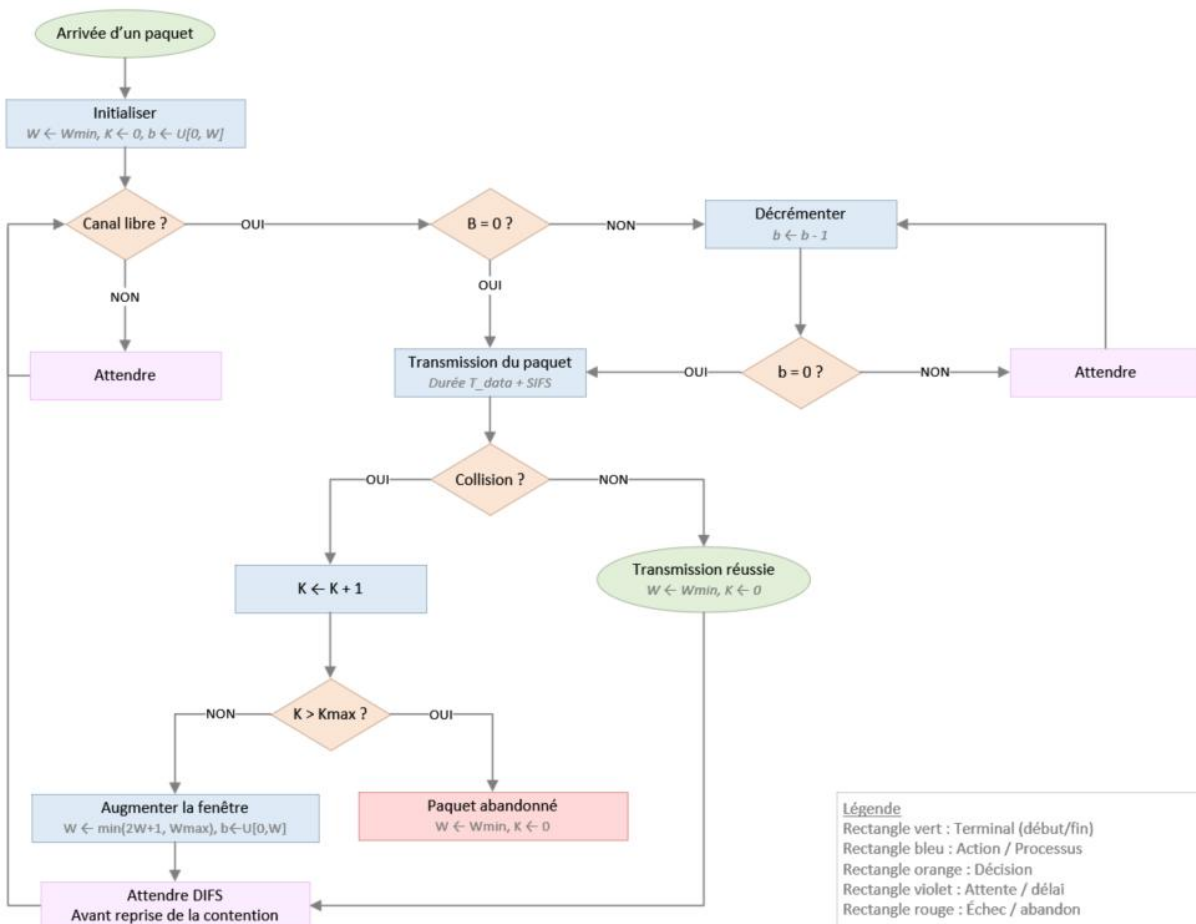
La logique du protocole CSMA/CA est intégrée directement dans le comportement des stations. Ce protocole repose sur quatre mécanismes fondamentaux définis par la fonction DCF, soit l'écoute du canal, l'algorithme d'attente binaire exponentielle, les espaces inter-trames (DIFS et SIFS) et les acquittements positifs.

Chaque station agit de manière autonome selon une prise de décision distribuée. Elle sélectionne un délai de backoff aléatoire b tiré uniformément dans l'intervalle $[0, W]$, puis surveille l'état du canal avant d'initier toute transmission. Cette écoute est modélisée de manière abstraite en supposant que le simulateur dispose d'une vue globale des états de toutes les stations, ce qui permet d'éviter une modélisation détaillée de la couche physique. Le compteur de backoff est décrémenté d'un slot à chaque intervalle de temps où le canal est libre. Une transmission est initiée lorsque ce compteur atteint zéro. En cas de transmission réussie, un acquittement positif est implicitement modélisé après un délai

SIFS, confirmant la bonne réception de la trame. L'absence d'acquittement est interprétée comme un échec de transmission, déclenchant le mécanisme de retransmission.

Une collision se produit lorsque plusieurs stations atteignent simultanément un compteur de backoff nul. Dans ce cas, chaque station impliquée incrémente son compteur de tentatives K et augmente sa fenêtre de contention selon la règle $W \leftarrow \min(2W + 1, W_{\max})$, puis attend une période DIFS avant de reprendre la contention avec un nouveau tirage de backoff. Si le nombre de tentatives dépasse $K_{\max}$, le paquet est abandonné et la station est réinitialisée à $W_{\min}$. Ce mécanisme adaptatif permet de réguler l'accès au canal et d'améliorer la stabilité du système en présence de forte contention.

Figure 2 — Fonctionnement du protocole CSMA/CA avec backoff exponentiel binaire



Cette figure illustre le flux décisionnel du protocole, mettant en évidence les étapes de sélection du backoff, de surveillance du canal, de transmission et de gestion des collisions.

Une extension intégrant le mécanisme RTS/CTS est également implémentée. Avant toute transmission de données, la station émettrice envoie une trame de demande d'émission (RTS). Si aucune collision RTS ne se produit, le point d'accès répond par une trame

d'autorisation d'émission (CTS), modélisée de manière abstraite dans le simulateur, puis la transmission des données est autorisée. Toutes les autres stations bloquent leur backoff via le vecteur d'allocation réseau (NAV) jusqu'à la fin prévue de la transmission de données. Ce mécanisme réduit les collisions sur les trames de données, au prix d'un surcoût introduit par les échanges de trames de contrôle supplémentaires.

3.4 Calcul des métriques de performance

Le simulateur intègre un module dédié à la collecte et à l'analyse des performances du réseau. Trois métriques principales sont calculées conformément aux définitions du sujet, auxquelles s'ajoutent des indicateurs complémentaires permettant d'obtenir une vision globale du comportement du système.

Le débit moyen correspond au nombre de bits transmis avec succès par unité de temps, exprimé en bits par seconde, et ne tient compte que des transmissions réussies :

$$\text{Débit} = \frac{(\text{Nombre de paquets réussis} \times L)}{T_{sim}}$$

où L est la taille d'un paquet en bits (12 000 bits, soit 1 500 octets, dans nos expériences) et T_{sim} est la durée totale de la simulation (20 secondes par défaut). Cette métrique reflète la capacité utile du canal, c'est-à-dire la fraction du débit physique effectivement utilisée pour transporter des données applicatives. Elle est directement influencée par le taux de collision et le temps passé en attente de backoff.

Le taux moyen de collision représente le rapport entre le nombre de tentatives de transmission ayant abouti à une collision et le nombre total de tentatives, exprimé en pourcentage :

$$\text{Taux de collision} = \frac{(\text{Nombre de transmissions en collision})}{(\text{Nombre total de tentatives})}$$

Cette métrique quantifie l'efficacité du mécanisme d'évitement des collisions. Un taux élevé indique que la fenêtre de contention est insuffisante pour espacer les tentatives des différentes stations, ce qui conduit à une dégradation du débit utile et à une augmentation du délai moyen.

Le délai moyen de transmission correspond au temps écoulé entre la génération d'un paquet et sa transmission réussie, exprimé en secondes (converti en millisecondes pour l'analyse des résultats) et moyenné sur l'ensemble des paquets transmis avec succès :

$$\text{Délai moyen} = \frac{(\sum(t_{succ} - t_{arr}))}{(\text{Nombre de paquets réussis})}$$

où t_{arr} est l'instant de génération du paquet et t_{succ} est l'instant de fin de transmission réussie. Ce délai inclut le temps d'attente lié au backoff, les éventuelles retransmissions ainsi que les périodes DIFS et SIFS. Il constitue un indicateur clé de la qualité de service perçue par les utilisateurs du réseau.

En complément, le simulateur calcule l'utilisation du canal, définie comme la fraction du temps pendant laquelle le médium transporte effectivement des données utiles :

$$U = \frac{(\text{Nombre de paquets réussis} \times T_{data})}{T_{sim}}$$

où T_{data} est la durée de transmission d'un paquet (1 ms par défaut, correspondant à un débit physique de 12 Mbps pour un paquet de 12 000 bits). Cette métrique se distingue du débit en ce qu'elle mesure l'occupation temporelle du canal plutôt que le volume de données transféré.

Enfin, le simulateur comptabilise également les paquets abandonnés, soit les paquets pour lesquels le nombre maximal de tentatives $K_{\{max\}}$ a été atteint sans succès. Lorsque ce seuil est dépassé, le paquet est définitivement perdu et la station est réinitialisée à $W_{\{min\}}$ pour le paquet suivant. Ce compteur permet d'évaluer le taux de perte sous forte charge et de mieux comprendre les limites du protocole dans des conditions de contention extrême.

En résumé, l'architecture du simulateur repose sur une séparation claire entre le moteur de simulation, le modèle des stations et le module de calcul des métriques. Cette organisation permet de reproduire de manière cohérente le fonctionnement du protocole CSMA/CA tout en facilitant l'étude de son comportement dans différents scénarios de charge.

4. Choix techniques

Le développement du simulateur repose sur plusieurs choix techniques visant à garantir la simplicité de l'implémentation, la clarté du modèle et la reproductibilité des résultats. Une attention particulière a été accordée à la modularité des composants afin de faciliter l'évolution du simulateur et l'intégration de nouveaux mécanismes sans remettre en cause l'architecture globale.

Le simulateur a été implémenté en Python, un langage largement utilisé en informatique scientifique et adapté au prototypage rapide. Les structures de données sont définies à l'aide de classes de données (dataclasses), ce qui améliore la lisibilité du code et réduit le code répétitif tout en facilitant la maintenance. Une compatibilité Python 3.8 et versions ultérieures a été assurée en gérant dynamiquement le paramètre slots des classes de données, absent dans les versions antérieures à Python 3.10.

La gestion des événements repose sur une file de priorité à tas minimal implémentée avec la bibliothèque `heapq`, garantissant un accès en temps logarithmique à l'événement le plus proche dans le temps. Chaque événement est associé à un horodatage et à un numéro de séquence monotone permettant de départager de manière déterministe les événements simultanés, assurant ainsi la reproductibilité des simulations. Un mécanisme de jeton permet d'invalidier les événements d'incrément de slot devenus obsolètes sans les retirer physiquement de la file, simplifiant ainsi la gestion des transitions d'état du canal.

Les choix de modélisation simplifient le système tout en conservant un comportement représentatif du protocole réel. L'écoute du canal est modélisée de manière abstraite en supposant que le simulateur dispose d'une vue globale des états des stations, ce qui correspond à un modèle centralisé et non à une représentation physique détaillée de la propagation du signal. Conformément aux hypothèses du sujet, chaque station ne traite qu'un seul paquet à la fois, ce qui limite le nombre de transmissions concurrentes et reflète le comportement de la couche MAC décrit par le standard IEEE 802.11.

La reproductibilité des résultats est assurée par l'utilisation d'un générateur aléatoire isolé (`random.Random`) initialisé avec une graine fixe, garantissant des simulations déterministes. La répétition des expériences avec calcul de la moyenne et de l'écart-type des résultats renforce la fiabilité statistique des tendances observées et permet de quantifier la variabilité inhérente au modèle.

Enfin, le simulateur offre un niveau élevé de configurabilité. L'ensemble des paramètres du protocole, soit le nombre de stations, le taux d'arrivée, $W_{\min}$, $W_{\max}$, $K_{\max}$, DIFS, SIFS, slot time et la durée de simulation, sont modifiables en ligne de commande, ce qui permet d'explorer facilement différents scénarios sans modifier le code source.

5. Résultats expérimentaux

Afin d'évaluer les performances du simulateur, plusieurs expériences ont été menées en faisant varier le nombre de stations, la charge du réseau et les paramètres du protocole. Les résultats sont présentés sous forme de graphiques illustrant l'évolution des trois métriques principales, soit le débit moyen, le taux de collision et le délai moyen de transmission. Chaque scénario a été exécuté plusieurs fois et les résultats ont été moyennés afin de réduire l'impact des fluctuations aléatoires.

5.1 Performance sous charge modérée ($\lambda = 20$ paquets/s par station)

La Figure 3 présente les résultats obtenus en fonction du nombre de stations dans les conditions de fonctionnement standard du protocole (taux d'arrivée de 20 paquets/s par station).

Figure 3 — Performances du protocole en fonction du nombre de stations (charge standard)

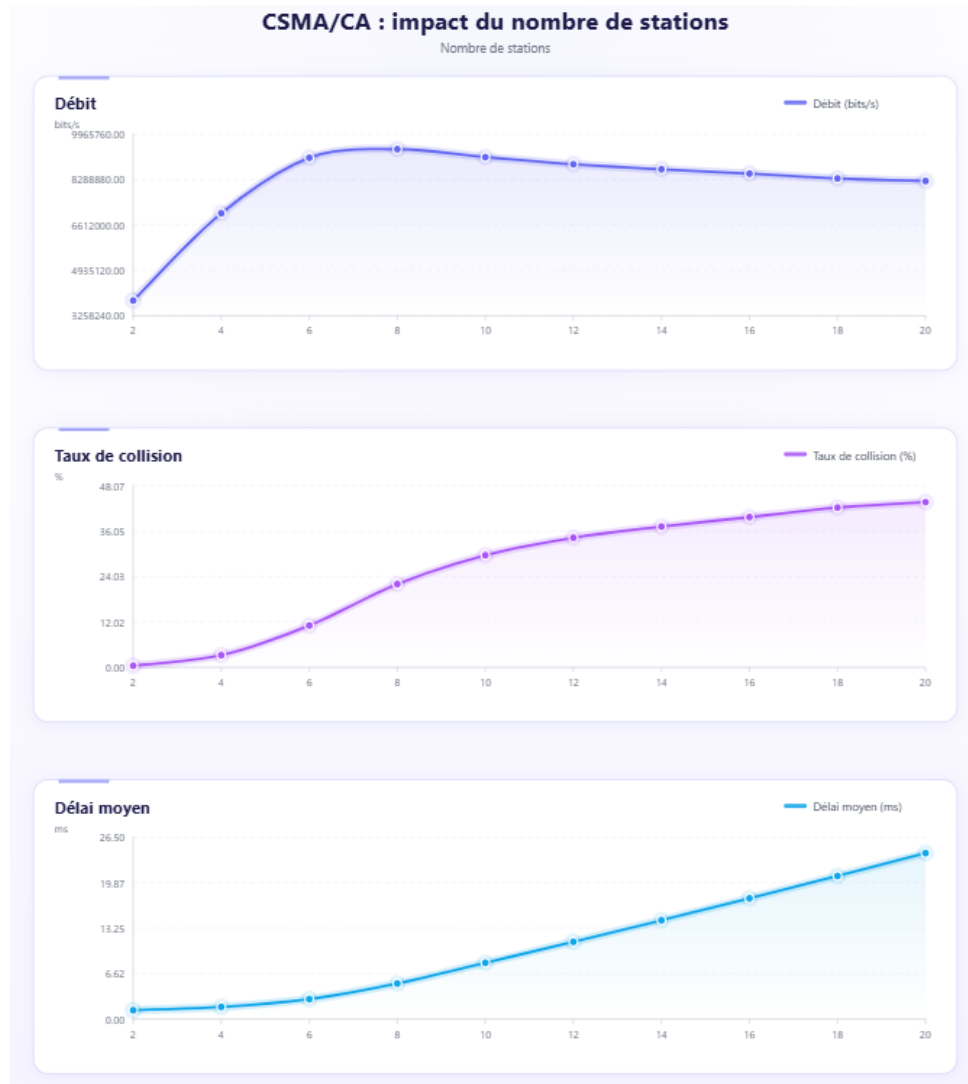


On observe que le débit augmente de manière quasi linéaire avec le nombre de stations, atteignant environ 4,60 Mbps pour 20 stations. Dans ce régime de charge modérée, le canal n'est jamais saturé et le protocole CSMA/CA gère efficacement la contention grâce au mécanisme d'attente binaire exponentielle. Le délai moyen reste faible, passant d'environ 1,17 ms à 1,61 ms, et le taux de collision demeure quasi nul (moins de 2,5 % pour 20 stations), confirmant que les collisions sont rares dans ce scénario. La stabilité de ces courbes reflète un fonctionnement fluide du protocole, dans lequel les mécanismes de régulation remplissent pleinement leur rôle.

5.2 Performance sous forte charge ($\lambda = 200$ paquets/s par station)

Afin d'étudier le comportement du système en situation de saturation, une seconde série d'expériences a été réalisée avec un taux d'arrivée nettement plus élevé (200 paquets/s par station). Les résultats sont présentés à la Figure 4.

Figure 4 — Performances du protocole en situation de forte charge



Dans ce scénario, le débit croît rapidement jusqu'à environ 8 stations, atteint un pic de 9,42 Mbps à $N=8$, puis décline progressivement jusqu'à 8,23 Mbps à $N=20$, traduisant une saturation rapide du canal. Le délai moyen augmente de façon exponentielle, passant de 1,29 ms à 2 stations jusqu'à 24,19 ms à 20 stations, en raison de l'algorithme d'attente binaire exponentielle qui force les stations à attendre des intervalles de plus en plus longs lorsque la contention est élevée. Le taux de collision augmente de façon monotone, atteignant 43,8 % pour 20 stations. La dégradation des performances provient donc de la

combinaison de la contention croissante et des collisions, et non d'un seul de ces facteurs isolément.

5.3 Impact de la taille minimale de la fenêtre de contention ($W_{\min}$)

La Figure 5 illustre l'effet de la valeur de $W_{\min}$ sur les performances du protocole, pour un réseau de 15 stations soumis à une forte charge.

Figure 5 — Impact de $W_{\min}$ sur les performances du protocole



Lorsque $W_{\min}$ est trop petit ($W_{\min}=3$), le taux de collision atteint 50,3 % et le délai est maximal (10,84 ms), car les stations tirent des délais d'attente très courts et entrent fréquemment en conflit. En augmentant $W_{\min}$, le débit croît et le taux de collision diminue de façon monotone. Le débit est maximisé autour de $W_{\min}=33$ (9,48 Mbps) et le délai est minimal autour de $W_{\min}=38$ (6,49 ms). Au-delà, le débit redéccline légèrement car les stations attendent inutilement longtemps même lorsque le canal est disponible. Ces

résultats confirment que la valeur standard IEEE 802.11b de $W_{\min}=15$ constitue un compromis conservateur, et que la zone optimale se situe entre 28 et 38 dans ce scénario.

5.4 Impact du nombre maximal de retransmissions ($K_{\max}$)

La Figure 6 présente l'impact de $K_{\max}$ sur les performances du protocole pour un réseau de 15 stations à forte charge (150 paquets/s par station).

Figure 6 — Impact de $K_{\max}$ sur les performances du protocole



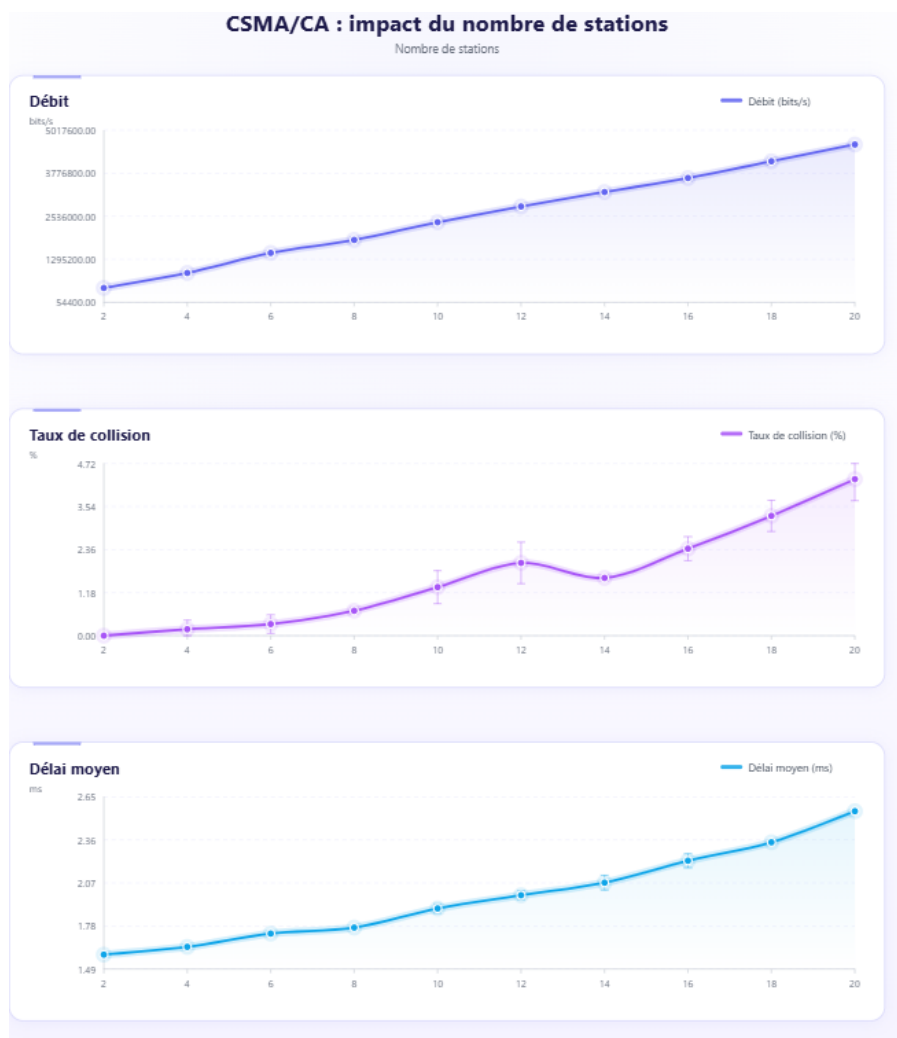
Pour $K_{\max}=1$, le débit est minimal (7,62 Mbps) et le délai est le plus faible (8,78 ms), car les paquets sont abandonnés rapidement après un seul échec, libérant le canal mais au prix d'un taux d'abandon élevé. En augmentant $K_{\max}$ de 1 à 6, le débit croît significativement et se stabilise autour de 8,65 Mbps, tandis que le délai augmente vers 13-14 ms et se stabilise également. Au-delà de $K_{\max}=6$, les métriques n'évoluent plus de façon

significative, car le protocole dispose de suffisamment de tentatives pour que l'algorithme d'attente binaire exponentielle converge sans pour autant accumuler des attentes inutiles. Ces résultats indiquent qu'une valeur de $K_{\max}$ comprise entre 6 et 8 constitue un bon compromis dans ce scénario.

5.5 Impact du mécanisme RTS/CTS

Une comparaison avec l'extension RTS/CTS a été effectuée dans les conditions de charge standard (20 paquets/s par station). Les résultats sont illustrés à la Figure 7.

Figure 7 — Performances du protocole avec mécanisme RTS/CTS



L'introduction du mécanisme RTS/CTS ajoute une phase de réservation du canal via le vecteur d'allocation réseau (NAV), ce qui protège les trames de données contre les collisions. Cependant, ce mécanisme introduit un surcoût lié aux échanges de trames de contrôle supplémentaires, réduisant le débit utile global à un maximum d'environ 4,60 Mbps pour 20 stations, inférieur au cas standard. Le délai de transmission augmente légèrement

(jusqu'à 2,56 ms) en raison de ces étapes supplémentaires. Par ailleurs, des collisions peuvent encore se produire au niveau des trames de demande d'émission elles-mêmes, atteignant un taux de 4,29 % pour 20 stations. Ainsi, dans un environnement simplifié sans terminal caché, l'utilisation de RTS/CTS ne permet pas d'améliorer les performances globales et introduit un compromis défavorable entre protection des données et surcoût protocolaire.

6. Analyse et interprétation des résultats

Les résultats obtenus permettent de mettre en évidence plusieurs tendances importantes concernant le comportement du protocole CSMA/CA et l'effet de ses paramètres.

En régime de charge standard, le débit augmente de manière quasi linéaire avec le nombre de stations, car le canal n'est jamais saturé et chaque station supplémentaire contribue directement au trafic utile. Le taux de collision demeure quasi nul (moins de 2,5 % pour 20 stations) et le délai reste faible (1,17 ms à 1,61 ms), ce qui confirme l'efficacité du mécanisme d'attente binaire exponentielle dans des conditions de charge modérée.

En régime de forte charge, le comportement est qualitativement différent. Le débit atteint un pic de 9,42 Mbps à $N=8$ puis décline progressivement jusqu'à 8,23 Mbps à $N=20$, traduisant une saturation rapide du canal. Le taux de collision augmente de façon significative, atteignant 43,8 % pour 20 stations, ce qui reflète la forte contention entre les stations. Parallèlement, le délai croît de façon exponentielle, passant de 1,29 ms à 2 stations jusqu'à 24,19 ms à 20 stations. Ce phénomène s'explique directement par l'algorithme d'attente binaire exponentielle, où à chaque collision, la fenêtre de contention évolue selon $W \leftarrow \min(2W+1, W_{\max})$, forçant les stations à attendre des intervalles de plus en plus longs. La dégradation des performances provient donc de la combinaison de la contention et des collisions croissantes, et non d'un seul de ces facteurs isolément. La comparaison entre les deux régimes illustre clairement la capacité finie du canal partagé à absorber un trafic croissant.

L'analyse de l'impact de $W_{\min}$ révèle l'existence d'une zone optimale clairement identifiable. Une valeur trop faible ($W_{\min}=3$) entraîne un taux de collision de 50,3 % et un délai de 10,84 ms, car les stations tirent des délais d'attente très courts et entrent fréquemment en conflit. Le débit est maximisé autour de $W_{\min}=33$ (9,48 Mbps) et le délai est minimal autour de $W_{\min}=38$ (6,49 ms). Au-delà, les performances redéclinent car les stations attendent inutilement longtemps même lorsque le canal est libre. Ces résultats montrent que la valeur standard IEEE 802.11b de $W_{\min}=15$ est un compromis conservateur, et que la zone optimale pour ce scénario se situe entre 28 et 38.

L'analyse de l'impact de $K_{\max}$ montre que ce paramètre influence principalement le délai et le débit, mais également le taux de collision, qui diminue de 51,3 % pour $K_{\max}=1$ à environ 38 % pour $K_{\max} \geq 6$, avant de se stabiliser. Pour $K_{\max}=1$, le débit est minimal (7,62 Mbps) et le délai est le plus faible (8,78 ms), car les paquets sont abandonnés rapidement après un seul échec. En augmentant $K_{\max}$ jusqu'à 6, le débit croît vers 8,65 Mbps et le délai se stabilise autour de 13-14 ms. Au-delà de $K_{\max}=6$, les métriques n'évoluent plus de façon significative, ce qui indique qu'une valeur entre 6 et 8 constitue un bon compromis dans ce scénario, soit suffisamment de tentatives pour que le mécanisme d'attente binaire exponentielle converge, sans accumulation d'attentes inutiles.

Concernant le mécanisme RTS/CTS, bien que celui-ci soit destiné à réduire les collisions sur les trames de données, il introduit une surcharge protocolaire en raison des échanges de trames de contrôle supplémentaires. Cela se traduit par un débit utile réduit (4,60 Mbps maximum) et un délai légèrement plus élevé (2,56 ms pour 20 stations) par rapport au cas standard. Les collisions observées concernent principalement les trames de demande d'émission elles-mêmes, atteignant 4,29 % pour 20 stations. Dans un environnement simplifié sans terminal caché, le bénéfice de ce mécanisme reste limité et ne compense pas le coût des transmissions supplémentaires, ce qui rejoint les remarques formulées dans les notes de cours concernant l'inefficacité de RTS/CTS en milieu à faible contention.

Dans l'ensemble, ces résultats mettent en évidence les compromis inhérents au protocole CSMA/CA, notamment entre efficacité du canal, délai de transmission, gestion de la contention et complexité protocolaire.

7. Conclusion

Ce projet a permis de développer un simulateur à événements discrets du protocole CSMA/CA, reproduisant les principaux mécanismes de gestion de l'accès au médium dans les réseaux sans fil. À travers cette implémentation, il a été possible de modéliser le comportement de plusieurs stations en compétition pour un canal partagé et d'évaluer les performances du système à l'aide de trois métriques principales, soit le débit moyen, le taux de collision et le délai moyen de transmission.

Les résultats expérimentaux mettent en évidence le fonctionnement global du protocole ainsi que ses limites. En régime de charge modérée, le mécanisme d'attente binaire exponentielle régule efficacement l'accès au canal et maintient un taux de collision quasi nul. En revanche, sous forte charge, le taux de collision peut atteindre 43,8 % et le délai croît de façon exponentielle jusqu'à 24,19 ms pour 20 stations, traduisant une dégradation significative des performances due à la contention croissante entre les stations.

L'analyse paramétrique a également permis d'identifier des zones optimales de fonctionnement. Pour $W_{\min}$, la zone optimale se situe entre 28 et 38 dans ce scénario, au-dessus de la valeur standard IEEE 802.11b de 15. Pour $K_{\max}$, une valeur entre 6 et 8 constitue un bon compromis entre débit et délai, au-delà duquel les gains deviennent négligeables.

L'étude du mécanisme RTS/CTS montre qu'il introduit un compromis entre la protection des trames de données et la surcharge protocolaire induite par les échanges de trames de contrôle supplémentaires. Dans un environnement simplifié sans terminal caché, ce mécanisme ne permet pas d'améliorer les performances globales et peut même réduire le débit utile, conformément aux observations formulées dans les notes de cours concernant les limites de ce mécanisme en milieu à faible contention.

Dans l'ensemble, ce projet a permis de mieux comprendre les principes de fonctionnement du protocole CSMA/CA et les enjeux liés au partage du médium dans les réseaux sans fil. Des améliorations futures pourraient inclure la modélisation de terminaux cachés, l'intégration d'un canal physique bruité ou l'extension du simulateur à des topologies multi-cellules afin d'explorer des scénarios plus réalistes.

Bibliographie

Python Software Foundation. Python Documentation. Consulté en 2026. Disponible à : <https://docs.python.org>

Sabir, E. Notes de cours INF1161 – Simulation et protocoles réseaux, Modules 4 et 5. Université TÉLUQ, consulté en 2026.

IEEE. (2020). IEEE Std 802.11-2020: Wireless LAN Medium Access Control (MAC) and Physical Layer Specifications. Disponible à : <https://ieeexplore.ieee.org/document/9363693>

Annexe

Annexe A –Présentation du dépôt GitHub

Le code source du simulateur ainsi que les outils nécessaires à la reproduction des résultats sont disponibles dans un dépôt GitHub dédié.

Le dépôt contient les éléments suivants :

- Le programme principal du simulateur CSMA/CA (`csma_ca_sim.py`), implémentant le mécanisme d'accès au médium avec attente binaire exponentielle et, en option, le mécanisme RTS/CTS
- Un script automatisé (`run_experiments.py`) permettant de reproduire l'ensemble des expériences présentées dans ce rapport
- Les fichiers de données générés à partir des simulations, enregistrés dans le répertoire `data/` sous forme de fichiers CSV
- Les graphiques produits automatiquement à partir des résultats des simulations, disponibles dans le répertoire `figures/`
- Un diagramme illustrant le fonctionnement du protocole (répertoire `assets/`)
- Un fichier de tests unitaires (`test_csma_ca_sim.py`), assurant la validation du comportement du simulateur et couvrant les principaux cas d'exécution
- Une documentation (`README.md`) décrivant l'installation, l'utilisation et les différentes options du simulateur
- Un fichier de dépendances (`requirements-dev.txt`) nécessaire à l'exécution des tests

Le dépôt est structuré de manière à assurer la reproductibilité complète des résultats présentés dans ce rapport. L'ensemble des expériences peut être reproduit automatiquement à l'aide de la commande suivante :

```
python run_experiments.py
```

Ce script exécute successivement les différentes configurations expérimentales, génère les fichiers de données dans le répertoire `data/` et produit les graphiques correspondants dans le répertoire `figures/`.

Lien vers le dépôt : [moyamelissa/Mobile-Networks-and-Wireless-Communications](https://github.com/moyamelissa/Mobile-Networks-and-Wireless-Communications)

Annexe B – Commandes de simulation et génération des graphiques

Cette annexe présente les différentes commandes utilisées pour générer les graphiques illustrant les performances du simulateur. Chaque commande permet de lancer une série de simulations en faisant varier certains paramètres, puis d'enregistrer les résultats sous forme de fichiers graphiques.

Toutes les simulations ont été réalisées en utilisant une graine fixe pour le générateur aléatoire, garantissant la reproductibilité des résultats.

1. Graphique principal (variation du nombre de stations)

Ce graphique permet d'observer l'évolution des performances du protocole CSMA/CA en fonction du nombre de stations dans des conditions de fonctionnement standard. Il est utilisé pour analyser l'impact de la charge progressive du réseau sur le débit, le délai et le taux de collision.

Commande :

```
py csma_ca_sim.py --sweep-stations 2 20 2 --runs 3 --simulation-time 5 --  
output stations.svg
```

Fichier généré : stations.svg

Explication de la commande :

- --sweep-stations 2 20 2 : fait varier le nombre de stations de 2 à 20 avec un pas de 2
- --runs 3 : exécute chaque simulation 3 fois afin de calculer une moyenne et réduire les fluctuations aléatoires
- --simulation-time 5 : définit la durée de chaque simulation à 5 secondes
- --output stations.svg : enregistre le graphique généré dans le fichier stations.svg

2. Graphique en charge élevée

Ce graphique permet d'étudier le comportement du protocole lorsque le réseau est soumis à une forte charge. Il met en évidence les phénomènes de saturation, notamment l'augmentation du délai et la stabilisation du débit.

Commande :

```
py csma_ca_sim.py --sweep-stations 2 20 2 --arrival-rate 60 --runs 3 --  
simulation-time 5 --output high_load.svg
```

Fichier généré : high_load.svg

Explication de la commande :

- `--sweep-stations 2 20 2` : fait varier le nombre de stations
- `--arrival-rate 60` : augmente le taux d'arrivée des paquets ($\lambda = 60$ paquets/s), correspondant à un scénario de charge élevée
- `--runs 3` : répète chaque simulation pour obtenir des résultats moyens plus fiables
- `--simulation-time 5` : fixe la durée de la simulation
- `--output high_load.svg` : génère le graphique correspondant dans le fichier *high_load.svg*

3. Graphique avec mécanisme RTS/CTS

Ce graphique permet d'évaluer l'impact du mécanisme RTS/CTS sur les performances du protocole. Il est utilisé pour comparer les résultats avec et sans réservation du canal.

Commande :

```
py csma_ca_sim.py --sweep-stations 2 20 2 --rtscts --runs 3 --simulation-time 5 --output rtscts.svg
```

Fichier généré : *rtscts.svg*

Explication de la commande :

- `--sweep-stations 2 20 2` : variation du nombre de stations
- `--rtscts` : active le mécanisme RTS/CTS dans le simulateur
- `--runs 3` : assure une moyenne sur plusieurs exécutions
- `--simulation-time 5` : durée de la simulation
- `--output rtscts.svg` : enregistre le graphique dans *rtscts.svg*

Annexe C - Code source du simulateur intégralement commenté

```
"""Simulateur à événements discrets du protocole CSMA/CA.

Ce module implémente la logique MAC du protocole CSMA/CA (Carrier Sense
Multiple Access with Collision Avoidance) tel que défini par le standard IEEE 802.11.
Il supporte le mode de base et une extension RTS/CTS avec mécanisme NAV.

Usage typique (ligne de commande) :
    python csma_ca_sim.py --stations 8 --arrival-rate 20 --simulation-time 20
    python csma_ca_sim.py --sweep-stations 2 20 2 --runs 3 --output
    resultats.svg

Structure principale :
    SimulationConfig - paramètres de la simulation (fenêtres,
    temporisations, etc.)
    StationState    - état MAC d'une station individuelle
    CSMACASimulator - moteur à événements discrets
    run_single_experiment / average_results / sweep_* - utilitaires
    d'expérimentation
    plot_points     - génération de graphiques SVG
    main           - point d'entrée en ligne de commande
"""
from __future__ import annotations

import argparse
import csv
import heapq
import math
import random
from dataclasses import dataclass as _dataclass

# Le paramètre `slots` de `dataclass` n'existe qu'à partir de Python 3.10.
# Cette compatibilité permet de garder un seul code source pour la CI en
# 3.8/3.9
# et les environnements locaux plus récents ; si `slots` n'est pas
# disponible,
# on le retire simplement.
_DATACLASS_SUPPORTS_SLOTS = True
try:
    # On crée une dataclass de test pour savoir si l'interpréteur gère
    # `slots`.
    _dataclass(slots=True)(type("_X", (), {}))
except TypeError: # pragma: no cover
    _DATACLASS_SUPPORTS_SLOTS = False # pragma: no cover

def dataclass_compat(**kwargs):
    """Crée un décorateur @dataclass compatible Python 3.8+ en retirant
    `slots` si nécessaire."""
    if not _DATACLASS_SUPPORTS_SLOTS and "slots" in kwargs:
        kwargs = {k: v for k, v in kwargs.items() if k != "slots"}
    return _dataclass(**kwargs)
from pathlib import Path
from statistics import mean, pstdev
from xml.sax.saxutils import escape
```

```

from typing import Optional

# -----
# Constantes – types d'événements du moteur de simulation
# -----
# Chaque constante identifie un type d'événement inséré dans la file de
# priorité.
# Utiliser des constantes nommées (plutôt que des entiers) rend le code
# lisible
# et facilite le débogage.
EVENT_ARRIVAL = "arrival"      # Arrivée d'un nouveau paquet dans une station
EVENT_SLOT_TICK = "slot_tick"  # Déclenchement d'un slot de backoff
EVENT_RTS_END = "rts_end"      # Fin de l'émission d'une trame RTS (mode
EVENT_DATA_END = "data_end"    # Fin de l'émission d'une trame de données

def next_slot_boundary(time_value: float, slot_time: float) -> float:
    """Retourne le prochain instant aligné sur une borne de slot.

    Le décalage de 1e-15 s évite qu'un instant déjà exactement aligné
    (flottant) soit arrondi au slot suivant à cause des erreurs de précision.

    Args:
        time_value: Instant courant (en secondes).
        slot_time:  Durée d'un slot (en secondes, doit être > 0).

    Returns:
        Le plus petit multiple de slot_time supérieur ou égal à time_value.

    Raises:
        ValueError: Si slot_time <= 0.
    """
    if slot_time <= 0:
        raise ValueError("slot_time must be positive")
    scaled = (time_value - 1e-15) / slot_time
    slot_index = math.ceil(scaled)
    return max(0.0, slot_index * slot_time)

@dataclass_compat(slots=True)
class SimulationConfig:
    """Paramètres globaux de la simulation.

    Regroupe toutes les hypothèses MAC et temporelles qui pilotent le modèle.
    Les valeurs par défaut correspondent au standard IEEE 802.11b.
    """
    station_count: int = 8          # Nombre de stations en compétition pour le
    canal
    arrival_rate: float = 20.0      # Taux d'arrivée par station (paquets/s) –
    processus de renouvellement
    simulation_time: float = 20.0   # Horizon de simulation (secondes)
    packet_bits: int = 12000        # Taille d'un paquet (bits) – 12 000 bits =
    1 500 octets
    packet_duration: float = 0.001  # Durée de transmission d'un paquet
    (secondes)

```

```

    slot_time: float = 20e-6      # Durée d'un slot de backoff (secondes)
    difs: float = 50e-6          # DIFS : délai inter-trames distribué
(secondes)
    sifs: float = 10e-6          # SIFS : délai inter-trames court
(secondes)
    wmin: int = 15                # Fenêtre de contention minimale W_min
    wmax: int = 1023              # Fenêtre de contention maximale W_max
    kmax: int = 15                # Nombre maximal de tentatives avant
abandon du paquet
    seed: Optional[int] = None    # Graine aléatoire (None = non
déterministe)
    rtscts: bool = False         # Active le mécanisme RTS/CTS et le NAV si
True
    rts_duration: float = 200e-6 # Durée d'une trame RTS (secondes)
    cts_duration: float = 200e-6 # Durée d'une trame CTS (secondes)

@dataclass_compat(slots=True)
class PacketState:
    """Représente une trame en attente de transmission dans une station.

    Une instance est créée à chaque arrivée de paquet et détruite après
    transmission réussie ou abandon (K > K_max).
    """
    arrival_time: float # Instant de génération du paquet – sert à calculer
le délai moyen
    attempts: int = 0   # Nombre de tentatives de transmission déjà
effectuées pour ce paquet

@dataclass_compat(slots=True)
class StationState:
    """État MAC d'une station individuelle.

    Le sujet impose qu'une station MAC ne garde qu'une seule trame à la fois
    :
    - `packet` est None quand la station est inactive, ou contient l'unique
trame
    en cours de transmission (attente de succès ou d'abandon).
    - `contention_window`, `backoff` et `retries` pilotent l'algorithme BEB.
    - `nav_until` matérialise la réservation du médium par le mécanisme NAV
(RTS/CTS).
    """
    station_id: int # Identifiant unique de la station
(index dans la liste)
    packet: Optional[PacketState] = None # Trame active (None = station
inactive)
    contention_window: int = 0 # Fenêtre de contention courante W ∈
[W_min, W_max]
    backoff: int = 0 # Compteur de backoff courant b ∈
[0, W]
    retries: int = 0 # Nombre de tentatives échouées pour
la trame courante (K)
    nav_until: float = 0.0 # Instant jusqu'auquel le médium est
réservé (NAV)

```



```

@dataclass_compat(slots=True)
class SimulationResult:
    """Résultats agrégés produits à la fin d'une simulation.

    Retourné par run_single_experiment() et average_results().
    Utilisé pour l'affichage console (print_result) et les graphiques
    (plot_points).
    """
    throughput_packets_per_s: float    # Débit en paquets transmis avec succès
    # par seconde
    throughput_bits_per_s: float       # Débit binaire utile (bits/s)
    channel_utilization: float         # Fraction du temps où le canal
    # transporte des données utiles
    offered_load_packets_per_s: float  # Charge offerte totale (paquets
    # générés / s)
    collision_rate: float               # Proportion des tentatives ayant
    # abouti à une collision
    mean_delay_s: float                # Délai moyen entre génération et
    # transmission réussie (s)
    generated_packets: int              # Nombre total de paquets générés
    # durant la simulation
    successful_packets: int             # Nombre de paquets transmis avec
    # succès
    dropped_packets: int                # Nombre de paquets abandonnés (K >
    # K_max)
    total_attempts: int                # Nombre total de tentatives de
    # transmission
    collided_packets: int               # Nombre de tentatives ayant abouti à
    # une collision

@dataclass_compat(slots=True)
class ExperimentPoint:
    """Un point de courbe expérimentale : une valeur de paramètre et les
    métriques associées.

    Produit par sweep_stations(), sweep_wmin() et sweep_kmax() ; consommé par
    plot_points().
    Les champs *_std contiennent l'écart-type calculé sur les `runs`
    répétitions,
    permettant de tracer des barres d'erreur ( $\pm 1\sigma$ ) sur les graphiques.
    """
    x_value: int                       # Valeur du paramètre balayé (ex.
    # nombre de stations)
    throughput_packets_per_s: float     # Débit moyen (paquets/s)
    throughput_bits_per_s: float        # Débit binaire moyen (bits/s)
    collision_rate: float               # Taux de collision moyen
    mean_delay_s: float                # Délai moyen de transmission
    # (secondes)
    throughput_bits_std: float = 0.0    # Écart-type du débit binaire entre
    # les répétitions (bits/s)
    collision_rate_std: float = 0.0     # Écart-type du taux de collision
    # entre les répétitions
    mean_delay_std: float = 0.0         # Écart-type du délai moyen entre les
    # répétitions (secondes)

```

```

class CSMACASimulator:
    """Moteur à événements discrets du protocole CSMA/CA.

    Gère la file de priorité des événements, l'état de chaque station,
    l'état global du canal et les compteurs de métriques.
    Instancier puis appeler run() pour obtenir un SimulationResult.
    """

    def __init__(self, config: SimulationConfig):
        """Initialise le simulateur avec la configuration donnée.

        Args:
            config: Paramètres de la simulation (voir SimulationConfig).
        """
        self.config = config
        # Générateur aléatoire isolé – garantit la reproductibilité via
        config.seed.
        self.random = random.Random(config.seed)
        # Une instance StationState par station participant à la contention.
        self.stations = [StationState(station_id=index) for index in
            range(config.station_count)]

        # File de priorité (min-heap) : chaque entrée est un tuple
        # (temps, sequence, type_événement, station_id, jeton).
        # Le numéro de séquence monotone brise les égalités de temps de façon
        déterministe.
        self.event_queue: list[tuple[float, int, str, int, Optional[int]]] =
            []
        self.sequence = 0 # Compteur global pour le numéro de séquence des
        événements

        # Ensemble des identifiants de stations actuellement en phase de
        backoff actif.
        self.contenders: set[int] = set()
        # Ensemble des stations engagées dans la transmission en cours (None
        si canal libre).
        # Si len > 1, une collision est en cours.
        self.current_transmission: Optional[set[int]] = None
        # Instant à partir duquel la contention peut reprendre (après DIFS).
        self.contention_open_time = 0.0
        # Instant planifié pour le prochain slot_tick (None si aucun
        planifié).
        self.scheduled_slot_tick_time: Optional[float] = None
        # Jeton permettant d'invalider les événements slot_tick périmés sans
        les retirer de la file.
        self.slot_tick_token = 0

        # Compteurs de métriques – cumulés au fil des événements, agrégés
        dans run().
        self.generated_packets = 0 # Paquets générés par les stations
        self.successful_packets = 0 # Paquets transmis avec succès
        self.dropped_packets = 0 # Paquets abandonnés après K_max
tentatives
        self.total_attempts = 0 # Tentatives de transmission totales
        self.collided_packets = 0 # Tentatives ayant abouti à une
collision

```

```

        self.successful_bits = 0      # Bits utiles transmis (pour le débit
binaire)
        self.delay_sum = 0.0         # Somme des délais individuels (pour la
moyenne)

    def run(self) -> SimulationResult:
        """Lance la simulation et retourne les métriques agrégées.

        1. Planifie la première arrivée de paquet pour chaque station.
        2. Traite les événements en ordre chronologique jusqu'à épuisement de
la file.
        3. Calcule et retourne les métriques de performance.

        Returns:
            SimulationResult contenant débit, taux de collision, délai moyen,
etc.
        """
        # Planification des premières arrivées : chaque station commence avec
        # un premier paquet tiré selon la loi exponentielle de paramètre
arrival_rate.
        for station_id in range(self.config.station_count):
            first_arrival = self._sample_interarrival()
            if first_arrival <= self.config.simulation_time:
                self._push_event(first_arrival, EVENT_ARRIVAL, station_id)

        # Boucle principale : dépile et traite chaque événement en ordre
chronologique.
        while self.event_queue:
            time_value, _, event_type, station_id, token =
heapq.heappop(self.event_queue)

            if event_type == EVENT_SLOT_TICK:
                if token != self.slot_tick_token:
                    continue # pragma: no cover
                self.scheduled_slot_tick_time = None

            if event_type == EVENT_ARRIVAL:
                self._handle_arrival(time_value, station_id)
            elif event_type == EVENT_SLOT_TICK:
                self._handle_slot_tick(time_value)
            elif event_type == EVENT_RTS_END:
                self._handle_rts_end(time_value)
            elif event_type == EVENT_DATA_END:
                self._handle_data_end(time_value)
            else:
                raise RuntimeError(f"Unknown event type: {event_type}")

        # --- Calcul des métriques finales ---
        # Débit en paquets et en bits : rapporte les succès à la durée de
simulation.
        throughput_packets_per_s = self.successful_packets /
self.config.simulation_time if self.config.simulation_time > 0 else 0.0
        throughput_bits_per_s = self.successful_bits /
self.config.simulation_time if self.config.simulation_time > 0 else 0.0
        # Utilisation du canal : fraction du temps effectivement occupé par
des données utiles.
        channel_utilization = (

```

```

        (self.successful_packets * self.config.packet_duration) /
self.config.simulation_time
        if self.config.simulation_time > 0 and
self.config.packet_duration > 0
            else 0.0
    )
    # Charge offerte : paquets générés par unité de temps (inclut succès
+ abandons).
    offered_load_packets_per_s = self.generated_packets /
self.config.simulation_time if self.config.simulation_time > 0 else 0.0
    # Taux de collision : proportion des tentatives ayant abouti à un
conflit.
    collision_rate = self.collided_packets / self.total_attempts if
self.total_attempts > 0 else 0.0
    # Délai moyen : moyenne des délais individuels (génération → succès).
    mean_delay_s = self.delay_sum / self.successful_packets if
self.successful_packets > 0 else 0.0

    return SimulationResult(
        throughput_packets_per_s=throughput_packets_per_s,
        throughput_bits_per_s=throughput_bits_per_s,
        channel_utilization=channel_utilization,
        offered_load_packets_per_s=offered_load_packets_per_s,
        collision_rate=collision_rate,
        mean_delay_s=mean_delay_s,
        generated_packets=self.generated_packets,
        successful_packets=self.successful_packets,
        dropped_packets=self.dropped_packets,
        total_attempts=self.total_attempts,
        collided_packets=self.collided_packets,
    )

def _push_event(self, time_value: float, event_type: str, station_id:
int, token: Optional[int] = None) -> None:
    """Insère un événement dans la file de priorité.

    Le numéro de séquence monotone garantit un ordre stable lorsque deux
événements partagent le même instant (tri secondaire déterministe).
    """
    self.sequence += 1
    heapq.heappush(self.event_queue, (time_value, self.sequence,
event_type, station_id, token))

def _sample_interarrival(self) -> float:
    """Tire un intervalle inter-arrivée selon la loi exponentielle de
paramètre arrival_rate.

    Les inter-arrivées sont exponentielles de paramètre  $\lambda = \text{arrival\_rate}$ ,
mais comme une station
    suspend la génération pendant le traitement d'un paquet, le processus
d'arrivée global
    est un processus de renouvellement – non un Poisson pur –
conformément à l'hypothèse du sujet.
    Retourne math.inf si arrival_rate <= 0, ce qui désactive les
arrivées.
    """
    if self.config.arrival_rate <= 0:

```

```

        return math.inf
    return self.random.expovariate(self.config.arrival_rate)

    def _sample_backoff(self, contention_window: int) -> int:
        """Tire un compteur de backoff b uniformément dans [0,
contention_window].

        Conforme à la règle IEEE 802.11 :  $b = U[0, W]$  où W est la fenêtre
courante.
        """
        return self.random.randint(0, contention_window)

    def _prime_station_for_contention(self, station_id: int, current_time:
float) -> None:
        """Initialise les compteurs d'une station et l'enregistre comme
contendante.

        Appelé à l'arrivée d'un nouveau paquet. Réinitialise W à W_min, K à
0,
        tire un backoff initial et déclenche le prochain slot_tick si le
canal est libre.
        """
        station = self.stations[station_id]
        if station.packet is None:
            return # Sécurité : ne rien faire si la station n'a pas de
paquet actif

        # Initialisation conforme à IEEE 802.11 : W_min et K=0 pour le
premier essai.
        station.contention_window = self.config.wmin
        station.retries = 0
        station.backoff = self._sample_backoff(station.contention_window)
        self.contenders.add(station_id)

        # Déclenche le slot_tick seulement si aucune transmission n'est en
cours.
        if self.current_transmission is None:
            self._schedule_slot_tick_if_needed(current_time)

    def _schedule_next_arrival(self, station_id: int, base_time: float) ->
None:
        """Planifie la prochaine arrivée de paquet pour une station.

        Conforme à l'hypothèse du sujet : la station ne génère un nouveau
paquet
        qu'après résolution (succès ou abandon) du paquet précédent.
        L'événement n'est pas planifié si l'instant calculé dépasse l'horizon
de simulation.
        """
        next_arrival = base_time + self._sample_interarrival()
        if next_arrival <= self.config.simulation_time:
            self._push_event(next_arrival, EVENT_ARRIVAL, station_id)

    def _schedule_slot_tick_if_needed(self, current_time: float) -> None:
        """Planifie le prochain événement slot_tick si les conditions le
permettent.

```

```

    Un slot_tick ne peut être planifié que si :
    - aucune transmission n'est en cours (canal libre), et
    - au moins une station est en phase de backoff actif.

    Le mécanisme de jeton (slot_tick_token) permet d'invalidier les
slot_ticks
    planifiés précédemment sans les retirer physiquement de la file de
priorité.
    Un slot_tick périmé (jeton obsolète) est simplement ignoré à la
dépile.
    """
    if self.current_transmission is not None or not self.contenders:
        return # Canal occupé ou aucune station en attente : pas de
slot_tick nécessaire

    # Le prochain slot doit respecter la période DIFS post-transmission
(contention_open_time).
    candidate = next_slot_boundary(max(current_time,
self.contention_open_time), self.config.slot_time)
    if self.scheduled_slot_tick_time is not None and
self.scheduled_slot_tick_time <= candidate:
        return # Un slot_tick déjà planifié à un instant antérieur est
suffisant

    # Invalide tout slot_tick précédent en incrémentant le jeton.
    self.slot_tick_token += 1
    self.scheduled_slot_tick_time = candidate
    self._push_event(candidate, EVENT_SLOT_TICK, -1,
self.slot_tick_token)

    def _start_transmission(self, current_time: float, station_ids:
list[int]) -> None:
        """Démarre une transmission de données (mode sans RTS/CTS).

        Si plusieurs stations transmettent simultanément (len > 1), c'est une
collision :
        la trame se termine sans SIFS car aucun ACK ne suivra.
        Si une seule station transmet, on ajoute le SIFS pour modéliser l'ACK
implicite.
        Dans tous les cas, un événement DATA_END est planifié en fin de
trame.
        """
        if not station_ids:
            return

        self.current_transmission = set(station_ids)
        for station_id in station_ids:
            self.contenders.discard(station_id) # La station quitte la phase
de contention
            self.stations[station_id].packet.attempts += 1 # type:
ignore[union-attr]

        self.total_attempts += len(station_ids)
        if len(station_ids) > 1:
            # Collision logique : plusieurs stations ont atteint b=0
simultanément.

```

```

        # Pas de SIFS car la trame est corrompue – aucun ACK ne sera
envoyé.
        release_time = current_time + self.config.packet_duration
    else:
        # Transmission unique réussie : on ajoute le SIFS (délai avant
ACK implicite).
        release_time = current_time + self.config.packet_duration +
self.config.sifs

        # Invalide tout slot_tick planifié : le canal est maintenant occupé.
self.slot_tick_token += 1
self.scheduled_slot_tick_time = None
self._push_event(release_time, EVENT_DATA_END, -1)

def _start_rts(self, current_time: float, station_ids: list[int]) ->
None:
    """Démarrage l'émission d'une trame RTS (mode RTS/CTS activé).

    Identique à _start_transmission mais planifie un événement RTS_END
    au lieu de DATA_END. La résolution collision/succès est traitée dans
_handle_rts_end, qui décidera ensuite si la donnée peut être
transmise.
    """
    if not station_ids:
        return

    self.current_transmission = set(station_ids)
    for station_id in station_ids:
        self contenders.discard(station_id) # La station entre en phase
RTS
        self.stations[station_id].packet.attempts += 1 # type:
ignore[union-attr]

    self.total_attempts += len(station_ids)
    rts_end = current_time + self.config.rts_duration
    # Invalide les slot_ticks pendant l'émission du RTS.
self.slot_tick_token += 1
self.scheduled_slot_tick_time = None
self._push_event(rts_end, EVENT_RTS_END, -1)

def _handle_arrival(self, current_time: float, station_id: int) -> None:
    """Traite l'arrivée d'un nouveau paquet dans une station.

    Crée une instance PacketState, incrémente le compteur global et
amorce
    la procédure de contention (initialisation de W, K et b) via
_prime_station_for_contention.
    Ignoré si la station possède déjà un paquet actif (ne devrait pas
arriver
    avec la génération interne, mais protège contre les cas
pathologiques).
    """
    station = self.stations[station_id]
    if station.packet is not None:
        # Cas théoriquement impossible avec la génération interne : la
station ne doit

```

```

        # pas produire une nouvelle trame tant que la précédente n'est
        pas résolue.
        return

    self.generated_packets += 1
    station.packet = PacketState(arrival_time=current_time)
    self._prime_station_for_contention(station_id, current_time)

    def _handle_slot_tick(self, current_time: float) -> None:
        """Traite un événement slot_tick : décrémente les backoffs et
        déclenche les transmissions.

        Logique en trois étapes :
        1. Si une ou plusieurs stations ont déjà b=0 avant décrémentation,
        elles transmettent.
        2. Sinon, décrémente b de 1 pour toutes les stations actives (hors
        NAV).
        3. Si après décrémentation une ou plusieurs stations atteignent b=0,
        elles transmettent.
        4. Sinon, planifie le slot suivant.

        La double vérification (avant et après décrémentation) gère
        correctement le cas
        où b=0 était déjà atteint lors du déclenchement initial de la
        contention.
        """
        if self.current_transmission is not None or not self.contenders:
            return # Canal occupé ou aucune station en contention : slot
            ignoré

        # Seules les stations hors NAV peuvent réellement contester le
        médium.
        # Les stations sous NAV (mode RTS/CTS) sont exclues de la
        décrémentation.
        active = [s for s in self.contenders if self.stations[s].nav_until <=
        current_time]

        # Étape 1 : stations déjà à b=0 – elles transmettent immédiatement.
        ready_to_send = [station_id for station_id in active if
        self.stations[station_id].backoff == 0]
        if ready_to_send:
            if self.config.rtscts:
                self._start_rts(current_time, ready_to_send)
            else:
                self._start_transmission(current_time, ready_to_send)
            return

        # Étape 2 : décrémentation du backoff pour toutes les stations
        actives.
        for station_id in active:
            station = self.stations[station_id]
            if station.backoff > 0:
                station.backoff -= 1

        # Étape 3 : stations qui atteignent b=0 après décrémentation.
        active_after = [station_id for station_id in active if
        self.stations[station_id].backoff == 0]

```



```

    if active_after:
        if self.config.rtscts:
            self._start_rts(current_time, active_after)
        else:
            self._start_transmission(current_time, active_after)
        return

    # Étape 4 : aucune station prête – planifie le slot suivant.
    self._schedule_slot_tick_if_needed(current_time +
self.config.slot_time)
    def _handle_rts_end(self, current_time: float) -> None:
        """Traite la fin d'une émission RTS (mode RTS/CTS).

        Deux cas :
        - Collision RTS (len > 1) : applique le BEB à chaque station
concernée,
          ou abandonne le paquet si K > K_max.
        - RTS réussi (len = 1) : le point d'accès envoie un CTS, toutes les
autres
          stations bloquent leur backoff via le NAV, puis la donnée est
transmise.
        """
        if self.current_transmission is None:
            return

        if len(self.current_transmission) > 1:
            # --- Collision RTS : plusieurs stations ont émis simultanément -
--
            affected_stations = list(self.current_transmission)
            self.collided_packets += len(affected_stations)
            self.current_transmission = None
            # Les stations doivent attendre DIFS avant de pouvoir re-
contester.
            self.contention_open_time = current_time + self.config.difs

            for station_id in affected_stations:
                station = self.stations[station_id]
                station.retries += 1
                if station.retries > self.config.kmax:
                    # K_max dépassé : paquet abandonné, réinitialisation
complète.
                    self.dropped_packets += 1
                    station.packet = None
                    station.contention_window = self.config.wmin
                    station.retries = 0
                    self._schedule_next_arrival(station_id, current_time)
                    continue

                # BEB :  $W \leftarrow \min(2W+1, W_{\max})$ , nouveau tirage de backoff.
                station.contention_window = min(2 * station.contention_window
+ 1, self.config.wmax)
                station.backoff =
self._sample_backoff(station.contention_window)
                self.contenders.add(station_id)

            self._schedule_slot_tick_if_needed(current_time)
            return

```

```

# --- RTS réussi : une seule station a émis ---
station_id = next(iter(self.current_transmission))
# Chronologie : RTS_end + SIFS + CTS + SIFS + DATA
data_end_time = current_time + self.config.sifs +
self.config.cts_duration + self.config.sifs + self.config.packet_duration

# Mécanisme NAV : toutes les autres stations bloquent leur backoff
# jusqu'à la fin prévue de la transmission de données.
for s in range(len(self.stations)):
    if s == station_id:
        continue
    st = self.stations[s]
    if st.packet is not None:
        st.nav_until = data_end_time # Réservation du médium via NAV

# On programme la fin de la transmission de données.
self._push_event(data_end_time, EVENT_DATA_END, -1)

def _handle_data_end(self, current_time: float) -> None:
    """Traite la fin d'une transmission de données.

    Deux cas :
    - Succès (len = 1) : enregistre le paquet, calcule le délai, libère
la station,
    ouvre la prochaine fenêtre de contention après DIFS.
    - Collision (len > 1) : applique le BEB à chaque station concernée
(mode sans RTS/CTS) ou abandonne si K > K_max.
    Avec RTS/CTS, ce cas est rare car le NAV protège la transmission.
    """
    if self.current_transmission is None:
        return

    is_collision = len(self.current_transmission) > 1
    if not is_collision:
        # --- Transmission réussie ---
        station_id = next(iter(self.current_transmission))
        station = self.stations[station_id]
        self.successful_packets += 1
        self.successful_bits += self.config.packet_bits
        # Délai individuel : temps entre génération et fin de
transmission réussie.
        self.delay_sum += current_time - station.packet.arrival_time #
type: ignore[union-attr]
        station.packet = None
        # Réinitialisation des compteurs de contention après succès.
        station.contention_window = self.config.wmin
        station.retries = 0
        self.current_transmission = None
        # La prochaine contention ne peut débuter qu'après DIFS.
        self.contention_open_time = current_time + self.config.difs
        self._schedule_next_arrival(station_id, current_time)
        self._schedule_slot_tick_if_needed(current_time)
        return

    # --- Collision de données : cas rare avec RTS/CTS, ou plusieurs
émetteurs synchrones ---

```

```

        affected_stations = list(self.current_transmission)
        self.collided_packets += len(affected_stations) # Chaque station
impliquée compte comme une collision
        self.current_transmission = None
        self.contention_open_time = current_time + self.config.difs

        for station_id in affected_stations:
            station = self.stations[station_id]
            station.retries += 1
            if station.retries > self.config.kmax:
                # K_max dépassé : paquet abandonné, réinitialisation
                self.dropped_packets += 1
                station.packet = None
                station.contention_window = self.config.wmin
                station.retries = 0
                self._schedule_next_arrival(station_id, current_time)
                continue

            # BEB :  $W \leftarrow \min(2W+1, W_{\max})$ , nouveau tirage de backoff.
            station.contention_window = min(2 * station.contention_window +
1, self.config.wmax)
            station.backoff = self._sample_backoff(station.contention_window)
            self.contenders.add(station_id)

        self._schedule_slot_tick_if_needed(current_time)

def run_single_experiment(config: SimulationConfig) -> SimulationResult:
    """Crée un simulateur, lance une simulation et retourne les métriques.

    Fonction utilitaire qui encapsule la création de CSMACASimulator et
    l'appel à run().
    """
    simulator = CSMACASimulator(config)
    return simulator.run()

def average_results(results: list[SimulationResult]) -> SimulationResult:
    """Calcule la moyenne de plusieurs SimulationResult.

    Les compteurs entiers (paquets générés, tentatives, etc.) sont sommés.
    Les métriques continues (débit, taux, délai) sont moyennées.
    Utilisé pour réduire les fluctuations statistiques lors de runs
    multiples.

    Raises:
        ValueError: Si la liste results est vide.
    """
    if not results:
        raise ValueError("results must not be empty")

    return SimulationResult(
        throughput_packets_per_s=mean(result.throughput_packets_per_s for
result in results),
        throughput_bits_per_s=mean(result.throughput_bits_per_s for result in
results),

```

```

        channel_utilization=mean(result.channel_utilization for result in
results),
        offered_load_packets_per_s=mean(result.offered_load_packets_per_s for
result in results),
        collision_rate=mean(result.collision_rate for result in results),
        mean_delay_s=mean(result.mean_delay_s for result in results),
        generated_packets=sum(result.generated_packets for result in
results),
        successful_packets=sum(result.successful_packets for result in
results),
        dropped_packets=sum(result.dropped_packets for result in results),
        total_attempts=sum(result.total_attempts for result in results),
        collided_packets=sum(result.collided_packets for result in results),
    )

```

```

def sweep_stations(
    base_config: SimulationConfig,
    start: int,
    stop: int,
    step: int,
    runs: int,
) -> list[ExperimentPoint]:
    """Balaye le nombre de stations de start à stop (inclus) par pas de step.

```

Pour chaque valeur, exécute `runs` simulations avec des graines différentes
et retourne la moyenne des métriques sous forme de liste d'ExperimentPoint.

Args:

- base_config: Configuration de référence (les autres paramètres sont conservés).
- start: Nombre de stations minimal.
- stop: Nombre de stations maximal (inclus).
- step: Pas d'incrémentation (doit être > 0).
- runs: Nombre de répétitions par configuration (doit être > 0).

Returns:

Liste d'ExperimentPoint triée par x_value croissant.

Raises:

ValueError: Si step <= 0 ou runs <= 0.

```

"""
if step <= 0:
    raise ValueError("step must be positive")
if runs <= 0:
    raise ValueError("runs must be positive")

```

```

points: list[ExperimentPoint] = []
for station_count in range(start, stop + 1, step):
    run_results: list[SimulationResult] = []
    for repeat_index in range(runs):
        config = SimulationConfig(
            station_count=station_count,
            arrival_rate=base_config.arrival_rate,
            simulation_time=base_config.simulation_time,

```

```

        packet_bits=base_config.packet_bits,
        packet_duration=base_config.packet_duration,
        slot_time=base_config.slot_time,
        difs=base_config.difs,
        sifs=base_config.sifs,
        wmin=base_config.wmin,
        wmax=base_config.wmax,
        kmax=base_config.kmax,
        seed=None if base_config.seed is None else base_config.seed +
repeat_index + station_count * 1000,
        rtscts=base_config.rtscts,
        rts_duration=base_config.rts_duration,
        cts_duration=base_config.cts_duration,
    )
    run_results.append(run_single_experiment(config))

    averaged = average_results(run_results)
    throughput_bits_std = pstdev(r.throughput_bits_per_s for r in
run_results)
    collision_rate_std = pstdev(r.collision_rate for r in run_results)
    mean_delay_std = pstdev(r.mean_delay_s for r in run_results)
    points.append(
        ExperimentPoint(
            x_value=station_count,
            throughput_packets_per_s=averaged.throughput_packets_per_s,
            throughput_bits_per_s=averaged.throughput_bits_per_s,
            throughput_bits_std=throughput_bits_std,
            collision_rate=averaged.collision_rate,
            collision_rate_std=collision_rate_std,
            mean_delay_s=averaged.mean_delay_s,
            mean_delay_std=mean_delay_std,
        )
    )

    return points

```

```

def sweep_wmin(
    base_config: SimulationConfig,
    start: int,
    stop: int,
    step: int,
    runs: int,
) -> list[ExperimentPoint]:
    """Balaye la fenêtre de contention minimale W_min de start à stop par pas
de step.

```

Identique à sweep_stations mais fait varier wmin au lieu du nombre de stations.

W_max est automatiquement ajusté à max(base_config.wmax, wmin) pour garantir wmin <= wmax à tout instant.

Args:

```

    base_config: Configuration de référence.
    start:      Valeur minimale de W_min.
    stop:      Valeur maximale de W_min (incluse).

```

```

        step:          Pas d'incrémentation (doit être > 0).
        runs:          Nombre de répétitions par configuration (doit être > 0).

Returns:
    Liste d'ExperimentPoint triée par x_value croissant.

Raises:
    ValueError: Si step <= 0 ou runs <= 0.
"""
if step <= 0:
    raise ValueError("step must be positive")
if runs <= 0:
    raise ValueError("runs must be positive")

points: list[ExperimentPoint] = []
for wmin in range(start, stop + 1, step):
    run_results: list[SimulationResult] = []
    for repeat_index in range(runs):
        config = SimulationConfig(
            station_count=base_config.station_count,
            arrival_rate=base_config.arrival_rate,
            simulation_time=base_config.simulation_time,
            packet_bits=base_config.packet_bits,
            packet_duration=base_config.packet_duration,
            slot_time=base_config.slot_time,
            difs=base_config.difs,
            sifs=base_config.sifs,
            wmin=wmin,
            wmax=max(base_config.wmax, wmin),
            kmax=base_config.kmax,
            seed=None if base_config.seed is None else base_config.seed +
repeat_index + wmin * 1000,
            rtscts=base_config.rtscts,
            rts_duration=base_config.rts_duration,
            cts_duration=base_config.cts_duration,
        )
        run_results.append(run_single_experiment(config))

    averaged = average_results(run_results)
    throughput_bits_std = pstdev(r.throughput_bits_per_s for r in
run_results)
    collision_rate_std = pstdev(r.collision_rate for r in run_results)
    mean_delay_std = pstdev(r.mean_delay_s for r in run_results)
    points.append(
        ExperimentPoint(
            x_value=wmin,
            throughput_packets_per_s=averaged.throughput_packets_per_s,
            throughput_bits_per_s=averaged.throughput_bits_per_s,
            throughput_bits_std=throughput_bits_std,
            collision_rate=averaged.collision_rate,
            collision_rate_std=collision_rate_std,
            mean_delay_s=averaged.mean_delay_s,
            mean_delay_std=mean_delay_std,
        )
    )

return points

```

```

def sweep_kmax(
    base_config: SimulationConfig,
    start: int,
    stop: int,
    step: int,
    runs: int,
) -> list[ExperimentPoint]:
    """Balaye le nombre maximal de tentatives K_max de start à stop par pas
    de step.

    Pour chaque valeur de K_max, exécute `runs` simulations avec des graines
    différentes
    et retourne la moyenne des métriques sous forme de liste
    d'ExperimentPoint.
    Un K_max faible provoque des abandons rapides (faible délai, paquets
    perdus) ;
    un K_max élevé laisse le BEB converger au prix d'un délai plus long.

    Args:
        base_config: Configuration de référence.
        start: Valeur minimale de K_max.
        stop: Valeur maximale de K_max (incluse).
        step: Pas d'incréméntation (doit être > 0).
        runs: Nombre de répétitions par configuration (doit être > 0).

    Returns:
        Liste d'ExperimentPoint triée par x_value croissant.

    Raises:
        ValueError: Si step <= 0 ou runs <= 0.
    """
    if step <= 0:
        raise ValueError("step must be positive")
    if runs <= 0:
        raise ValueError("runs must be positive")

    points: list[ExperimentPoint] = []
    for kmax in range(start, stop + 1, step):
        run_results: list[SimulationResult] = []
        for repeat_index in range(runs):
            config = SimulationConfig(
                station_count=base_config.station_count,
                arrival_rate=base_config.arrival_rate,
                simulation_time=base_config.simulation_time,
                packet_bits=base_config.packet_bits,
                packet_duration=base_config.packet_duration,
                slot_time=base_config.slot_time,
                difs=base_config.difs,
                sifs=base_config.sifs,
                wmin=base_config.wmin,
                wmax=base_config.wmax,
                kmax=kmax,
                seed=None if base_config.seed is None else base_config.seed +
                repeat_index + kmax * 1000,
                rtscts=base_config.rtscts,

```

```

        rts_duration=base_config.rts_duration,
        cts_duration=base_config.cts_duration,
    )
    run_results.append(run_single_experiment(config))

    averaged = average_results(run_results)
    throughput_bits_std = pstdev(r.throughput_bits_per_s for r in
run_results)
    collision_rate_std = pstdev(r.collision_rate for r in run_results)
    mean_delay_std = pstdev(r.mean_delay_s for r in run_results)
    points.append(
        ExperimentPoint(
            x_value=kmax,
            throughput_packets_per_s=averaged.throughput_packets_per_s,
            throughput_bits_per_s=averaged.throughput_bits_per_s,
            throughput_bits_std=throughput_bits_std,
            collision_rate=averaged.collision_rate,
            collision_rate_std=collision_rate_std,
            mean_delay_s=averaged.mean_delay_s,
            mean_delay_std=mean_delay_std,
        )
    )

    return points

def print_result(config: SimulationConfig, result: SimulationResult) -> None:
    """Affiche la configuration et les métriques de simulation dans la
    console.

    Produit un bloc lisible avec les paramètres utilisés et les résultats
    obtenus.
    """
    print("Configuration")
    print(f"  Stations : {config.station_count}")
    print(f"  Arrival rate/station : {config.arrival_rate:.4f} packets/s")
    print(f"  Simulated time : {config.simulation_time:.4f} s")
    print(f"  Packet duration : {config.packet_duration:.6f} s")
    print(f"  Packet size : {config.packet_bits} bits")
    print(f"  Slot time : {config.slot_time:.8f} s")
    print(f"  DIFS / SIFS : {config.difs:.8f} s / {config.sifs:.8f} s")
    print(f"  Wmin / Wmax / Kmax : {config.wmin} / {config.wmax} / {config.kmax}")
    print()
    print("Results")
    print(f"  Throughput : {result.throughput_packets_per_s:.4f} packets/s")
    print(f"  Throughput : {result.throughput_bits_per_s:.4f} bits/s")
    print(f"  Channel utilization : {result.channel_utilization * 100:.2f} %")
    print(f"  Offered load : {result.offered_load_packets_per_s:.4f} packets/s")
    print(f"  Mean collision rate : {result.collision_rate * 100:.2f} %")
    print(f"  Mean transmission delay: {result.mean_delay_s * 1000:.4f} ms")
    print(f"  Generated packets : {result.generated_packets}")

```



```

print(f"   Successful packets      : {result.successful_packets}")
print(f"   Dropped packets           : {result.dropped_packets}")
print(f"   Total attempts             : {result.total_attempts}")
print(f"   Collided packets           : {result.collided_packets}")

def save_csv(
    points: list[ExperimentPoint] | None,
    result: SimulationResult | None,
    config: SimulationConfig | None,
    csv_path: Path,
) -> None:
    """Sauvegarde les résultats bruts dans un fichier CSV.

    Deux modes :
    - sweep (points non vide) : une ligne par valeur du paramètre balayé,
      avec les métriques moyennes et les écarts-types.
    - simulation unique (result/config non nuls) : une seule ligne avec
      tous les compteurs et métriques de SimulationResult.
    """
    csv_path.parent.mkdir(parents=True, exist_ok=True)
    if points:
        fieldnames = [
            "x_value",
            "throughput_packets_per_s",
            "throughput_bits_per_s",
            "throughput_bits_std",
            "collision_rate",
            "collision_rate_std",
            "mean_delay_s",
            "mean_delay_std",
        ]
        with csv_path.open("w", newline="", encoding="utf-8") as fh:
            writer = csv.DictWriter(fh, fieldnames=fieldnames)
            writer.writeheader()
            for p in points:
                writer.writerow({
                    "x_value": p.x_value,
                    "throughput_packets_per_s": p.throughput_packets_per_s,
                    "throughput_bits_per_s": p.throughput_bits_per_s,
                    "throughput_bits_std": p.throughput_bits_std,
                    "collision_rate": p.collision_rate,
                    "collision_rate_std": p.collision_rate_std,
                    "mean_delay_s": p.mean_delay_s,
                    "mean_delay_std": p.mean_delay_std,
                })
    elif result is not None and config is not None:
        fieldnames = [
            "station_count",
            "arrival_rate",
            "wmin",
            "wmax",
            "kmax",
            "throughput_packets_per_s",
            "throughput_bits_per_s",
            "channel_utilization",
            "offered_load_packets_per_s",

```

```

        "collision_rate",
        "mean_delay_s",
        "generated_packets",
        "successful_packets",
        "dropped_packets",
        "total_attempts",
        "collided_packets",
    ]
    with csv_path.open("w", newline="", encoding="utf-8") as fh:
        writer = csv.DictWriter(fh, fieldnames=fieldnames)
        writer.writeheader()
        writer.writerow({
            "station_count": config.station_count,
            "arrival_rate": config.arrival_rate,
            "wmin": config.wmin,
            "wmax": config.wmax,
            "kmax": config.kmax,
            "throughput_packets_per_s": result.throughput_packets_per_s,
            "throughput_bits_per_s": result.throughput_bits_per_s,
            "channel_utilization": result.channel_utilization,
            "offered_load_packets_per_s":
result.offered_load_packets_per_s,
            "collision_rate": result.collision_rate,
            "mean_delay_s": result.mean_delay_s,
            "generated_packets": result.generated_packets,
            "successful_packets": result.successful_packets,
            "dropped_packets": result.dropped_packets,
            "total_attempts": result.total_attempts,
            "collided_packets": result.collided_packets,
        })

```

```

def plot_points(points: list[ExperimentPoint], title: str, x_label: str,
output_path: Path) -> None:
    """Génère un graphique SVG à trois panneaux (débit, taux de collision,
délai moyen).

```

Le SVG est auto-contenu (pas de dépendance externe) et s'affiche directement dans un navigateur ou peut être intégré dans un rapport PDF.

```

Args:
    points:      Liste de points expérimentaux (un par valeur de
paramètre).
    title:       Titre principal affiché en haut du graphique.
    x_label:     Libellé de l'axe des abscisses commun aux trois
panneaux.
    output_path: Chemin du fichier SVG de sortie (les dossiers sont créés
si nécessaire).

```

```

Raises:
    ValueError: Si points est vide.
    """
    if not points:
        raise ValueError("points must not be empty")

```

```

width = 980

```

```

height = 1190
panel_width = 880
panel_height = 280
left = 50
top_margin = 90
panel_gap = 55
inner_left = 95
inner_right = 30
inner_top = 55
inner_bottom = 52

# Blue/purple light-theme palette: (stroke, area-top-opacity, area-bot-
opacity)
PALETTE = [
    ("#6366f1", "0.14", "0.0"), # indigo - throughput
    ("#a855f7", "0.12", "0.0"), # purple - collision
    ("#0ea5e9", "0.11", "0.0"), # sky - delay
]

x_values = [point.x_value for point in points]
throughput_bits = [point.throughput_bits_per_s for point in points]
collision_rates = [point.collision_rate * 100 for point in points]
mean_delays = [point.mean_delay_s * 1000 for point in points]
throughput_bits_stds = [point.throughput_bits_std for point in points]
collision_stds = [point.collision_rate_std * 100 for point in points]
delay_stds = [point.mean_delay_std * 1000 for point in points]

def scale_x(index: int, count: int) -> float:
    """Convertit l'indice d'un point en coordonnée X SVG.

    Distribue uniformément les points sur la largeur utile du panneau.
    Retourne le centre horizontal si count == 1 (un seul point de
données).
    """
    plot_w = panel_width - inner_left - inner_right
    if count == 1:
        return left + inner_left + plot_w / 2
    return left + inner_left + (plot_w * index / (count - 1))

def scale_y(value: float, minimum: float, maximum: float, panel_top:
float) -> float:
    """Convertit une valeur de données en coordonnée Y SVG (axe inversé :
haut = maximum).

    Retourne le centre vertical du panneau si minimum == maximum (axe
dégénéré),
    afin d'éviter une division par zéro.
    """
    plot_h = panel_height - inner_top - inner_bottom
    if math.isclose(minimum, maximum): # pragma: no cover
        return panel_top + inner_top + plot_h / 2 # pragma: no cover
    return panel_top + inner_top + (maximum - value) * plot_h / (maximum
- minimum)

def format_ticks(minimum: float, maximum: float, count: int = 5) ->
list[float]:

```

```

        """Calcule les valeurs des graduations régulièrement espacées sur
l'axe Y.

    Retourne `count` valeurs de minimum à maximum inclus.
    Si minimum == maximum (axe dégénéré), retourne [minimum] pour éviter
une erreur.
    """
    if math.isclose(minimum, maximum): # pragma: no cover
        return [minimum] # pragma: no cover
    step = (maximum - minimum) / (count - 1)
    return [minimum + step * index for index in range(count)]

def smooth_curve(coords: list[tuple[float, float]]) -> str:
    """Génère un chemin SVG lissé à partir d'une liste de points (x, y).

    Utilise l'algorithme de spline de Catmull-Rom converti en courbes de
Bézier
    cubiques : les tangentes en chaque point sont calculées à partir des
voisins
    (P_{i-1} et P_{i+1}), ce qui produit une courbe C1 continue et
visuellement
    douce sans nécessiter de bibliothèque externe.
    Retourne la chaîne de commande SVG au format « M... C... ».
    """
    if len(coords) < 2:
        return f"M{coords[0][0]:.2f},{coords[0][1]:.2f}"
    n = len(coords)
    parts = [f"M{coords[0][0]:.2f},{coords[0][1]:.2f}"]
    for i in range(n - 1):
        p0 = coords[max(0, i - 1)]
        p1 = coords[i]
        p2 = coords[i + 1]
        p3 = coords[min(n - 1, i + 2)]
        cp1x = p1[0] + (p2[0] - p0[0]) / 6
        cp1y = p1[1] + (p2[1] - p0[1]) / 6
        cp2x = p2[0] - (p3[0] - p1[0]) / 6
        cp2y = p2[1] - (p3[1] - p1[1]) / 6
        parts.append(f"C{cp1x:.2f},{cp1y:.2f} {cp2x:.2f},{cp2y:.2f}
{p2[0]:.2f},{p2[1]:.2f}")
    return " ".join(parts)

# Defs are accumulated here; panel_svg appends per-series area gradients.
defs_parts: list[str] = [
    '<linearGradient id="svgBg" x1="0" y1="0" x2="0" y2="1"
gradientUnits="objectBoundingBox">',
    ' <stop offset="0%" stop-color="#ffffff"/>',
    ' <stop offset="100%" stop-color="#f5f3ff"/>',
    '</linearGradient>',
    '<radialGradient id="topGlow" cx="50%" cy="0%" r="60%">',
    ' <stop offset="0%" stop-color="#6366f1" stop-opacity="0.05"/>',
    ' <stop offset="100%" stop-color="#6366f1" stop-opacity="0"/>',
    '</radialGradient>',
]

def panel_svg(panel_index: int, panel_top: float, panel_title: str,
y_label: str,

```

```

        series: list[tuple[list[float], list[float], int, str]]) ->
str:
    """Génère le SVG complet d'un panneau de graphique pour une métrique
donnée.

    Trace la carte de fond, les axes, les lignes de grille, les courbes
lissées,
    les barres d'erreur ( $\pm 1\sigma$ ) et les points de données pour chaque série
fournie.
    Les dégradés de zone sont enregistrés dans `defs_parts` (accessible
en closure).

    Args:
        panel_index: Indice du panneau (0=débit, 1=collision, 2=délai) -
utilisé
                    pour générer des identifiants SVG uniques
                    (gradients, etc.).
        panel_top:   Coordonnée Y du bord supérieur du panneau (pixels
SVG).
        panel_title: Titre affiché en haut à gauche du panneau.
        y_label:     Libellé de l'unité affiché sous le titre (ex.
"bits/s", "%", "ms").
        series:      Liste de séries à tracer ; chaque élément est un
tuple
                    (valeurs, écarts-types, indice_palette, légende).

    Returns:
        Chaîne SVG (sans balise racine) représentant le panneau complet.
    """
    y_min = min(min(values) for values, _, _ in series)
    y_max = max(max(values) for values, _, _ in series)
    if math.isclose(y_min, y_max):
        y_min = 0.0
        y_max = y_max + 1.0
    y_padding = (y_max - y_min) * 0.10 or 1.0
    y_min = max(0.0, y_min - y_padding)
    y_max = y_max + y_padding

    plot_left = left + inner_left
    plot_top = panel_top + inner_top
    plot_w = panel_width - inner_left - inner_right
    plot_h = panel_height - inner_top - inner_bottom
    plot_bottom = plot_top + plot_h

    el: list[str] = []

    # Card
    el.append(
        f'<rect x="{left}" y="{panel_top}" width="{panel_width}"
height="{panel_height}"'
        f' rx="14" fill="#ffffff" stroke="rgba(99,102,241,0.18)" stroke-
width="1.2"'
        f' filter="drop-shadow(0 2px 8px rgba(99,102,241,0.08))"/>'
    )
    # Accent top bar
    el.append(
        f'<rect x="{left + 24}" y="{panel_top}" width="48" height="3"'

```

```

        f' rx="2" fill="rgba(99,102,241,0.55)" />'
    )
    # Panel title
    el.append(
        f'<text x="{left + 20}" y="{panel_top + 32}"'
        f' font-family="system-ui,\''Segoe UI\','Arial,sans-serif"'
        f' font-size="15" font-weight="700" fill="#1e1b4b" letter-
spacing="0.3">'
        f'{escape(panel_title)}</text>'
    )
    # y-label (unit)
    el.append(
        f'<text x="{left + 20}" y="{panel_top + 50}"'
        f' font-family="system-ui,\''Segoe UI\','Arial,sans-serif"'
        f' font-size="10" fill="#6b7280">{escape(y_label)}</text>'
    )

    # Horizontal grid lines
    for tick_value in format_ticks(y_min, y_max):
        y = scale_y(tick_value, y_min, y_max, panel_top)
        el.append(
            f'<line x1="{plot_left}" y1="{y:.2f}" x2="{plot_left +
plot_w}" y2="{y:.2f}"'
            f' stroke="rgba(99,102,241,0.08)" stroke-width="1" stroke-
dasharray="4,4" />'
        )

    # Axes
    el.append(
        f'<line x1="{plot_left}" y1="{plot_bottom}" x2="{plot_left +
plot_w}" y2="{plot_bottom}"'
        f' stroke="#d1d5db" stroke-width="1" />'
    )
    el.append(
        f'<line x1="{plot_left}" y1="{plot_top}" x2="{plot_left}"
y2="{plot_bottom}"'
        f' stroke="#d1d5db" stroke-width="1" />'
    )

    # Y-axis labels
    for tick_value in format_ticks(y_min, y_max):
        y = scale_y(tick_value, y_min, y_max, panel_top)
        el.append(
            f'<text x="{plot_left - 8}" y="{y + 4:.2f}" text-
anchor="end"'
            f' font-family="system-ui,\''Segoe UI\','Arial,sans-serif"'
            f' font-size="10" fill="#6b7280">{tick_value:.2f}</text>'
        )

    # X-axis labels
    for idx, x_value in enumerate(x_values):
        x = scale_x(idx, len(x_values))
        el.append(
            f'<line x1="{x:.2f}" y1="{plot_bottom}" x2="{x:.2f}"
y2="{plot_bottom + 4}"'
            f' stroke="#d1d5db" stroke-width="1" />'
        )

```

```

        el.append(
            f'<text x="{x:.2f}" y="{plot_bottom + 18}" text-
anchor="middle"'
            f' font-family="system-ui,\''Segoe UI\','Arial,sans-serif"'
            f' font-size="10" fill="#6b7280">{x_value}</text>'
        )

    # Per-series rendering
    for si, (values, stds, ci, label) in enumerate(series):
        stroke_color = PALETTE[ci][0]
        area_op_top = PALETTE[ci][1]
        area_op_bot = PALETTE[ci][2]

        coords = [
            (scale_x(idx, len(x_values)), scale_y(v, y_min, y_max,
panel_top))
            for idx, v in enumerate(values)
        ]

        # Register area gradient in defs
        grad_id = f"areaGrad_p{panel_index}_s{si}"
        defs_parts.append(
            f'<linearGradient id="{grad_id}" x1="0" y1="{plot_top:.2f}"
x2="0" y2="{plot_bottom:.2f}"'
            f' gradientUnits="userSpaceOnUse">'
        )
        defs_parts.append(
            f' <stop offset="0%" stop-color="{stroke_color}" stop-
opacity="{area_op_top}">'
        )
        defs_parts.append(
            f' <stop offset="100%" stop-color="{stroke_color}" stop-
opacity="{area_op_bot}">'
        )
        defs_parts.append('</linearGradient>')

        # Area fill
        curve_d = smooth_curve(coords)
        area_d = (
            f'{curve_d}'
            f' L{coords[-1][0]:.2f},{plot_bottom:.2f}'
            f' L{coords[0][0]:.2f},{plot_bottom:.2f} Z'
        )
        el.append(f'<path d="{area_d}" fill="url("#{grad_id})"
stroke="none"/>')

        # Glow halo behind the line
        el.append(
            f'<path d="{curve_d}" fill="none" stroke="{stroke_color}"'
            f' stroke-width="8" stroke-opacity="0.18"'
            f' stroke-linecap="round" stroke-linejoin="round"/>'
        )
        # Main line
        el.append(
            f'<path d="{curve_d}" fill="none" stroke="{stroke_color}"'
            f' stroke-width="2.5" stroke-linecap="round" stroke-
linejoin="round"/>'

```

```

    )

    # Error bars ( $\pm 1 \sigma$ )
    for idx, (value, std) in enumerate(zip(values, stds)):
        if std > 0:
            x = scale_x(idx, len(x_values))
            y_hi = scale_y(min(value + std, y_max), y_min, y_max,
panel_top)
            y_lo = scale_y(max(value - std, y_min), y_min, y_max,
panel_top)
            el.append(
                f'<line x1="{x:.2f}" y1="{y_hi:.2f}" x2="{x:.2f}"
y2="{y_lo:.2f}"'
                f' stroke="{stroke_color}" stroke-width="1.2" stroke-
opacity="0.45"/>'
            )
            el.append(
                f'<line x1="{x-4:.2f}" y1="{y_hi:.2f}" x2="{x+4:.2f}"
y2="{y_hi:.2f}"'
                f' stroke="{stroke_color}" stroke-width="1.2" stroke-
opacity="0.45"/>'
            )
            el.append(
                f'<line x1="{x-4:.2f}" y1="{y_lo:.2f}" x2="{x+4:.2f}"
y2="{y_lo:.2f}"'
                f' stroke="{stroke_color}" stroke-width="1.2" stroke-
opacity="0.45"/>'
            )

    # Data dots: outer glow ring + filled core
    for idx, value in enumerate(values):
        x = scale_x(idx, len(x_values))
        y = scale_y(value, y_min, y_max, panel_top)
        el.append(
            f'<circle cx="{x:.2f}" cy="{y:.2f}" r="8"'
            f' fill="{stroke_color}" fill-opacity="0.12"
stroke="none"/>'
        )
        el.append(
            f'<circle cx="{x:.2f}" cy="{y:.2f}" r="4"'
            f' fill="{stroke_color}" stroke="#ffffff" stroke-
width="1.5"/>'
        )

    # Legend entry
    legend_x = left + panel_width - 165
    legend_y = panel_top + 32 + si * 22
    el.append(
        f'<rect x="{legend_x}" y="{legend_y - 9}" width="22"
height="4"'
        f' rx="2" fill="{stroke_color}" fill-opacity="0.85"/>'
    )
    el.append(
        f'<text x="{legend_x + 28}" y="{legend_y - 2}"'
        f' font-family="system-ui, \'Segoe UI\', Arial, sans-serif"'
        f' font-size="11" fill="#374151">{escape(label)}</text>'
    )

```



```

        return "\n".join(el)

    throughput_panel = panel_svg(
        0, top_margin,
        "Débit", "bits/s",
        [(throughput_bits, throughput_bits_stds, 0, "Débit (bits/s)"]],
    )
    collision_panel = panel_svg(
        1, top_margin + panel_height + panel_gap,
        "Taux de collision", "%",
        [(collision_rates, collision_stds, 1, "Taux de collision (%)"]],
    )
    delay_panel = panel_svg(
        2, top_margin + (panel_height + panel_gap) * 2,
        "Délai moyen", "ms",
        [(mean_delays, delay_stds, 2, "Délai moyen (ms)"]],
    )

    defs_xml = "\n    ".join(defs_parts)
    svg = f'<svg xmlns="http://www.w3.org/2000/svg" width="{width}"
height="{height}" viewBox="0 0 {width} {height}">
    <defs>
    {defs_xml}
    </defs>
    <rect width="100%" height="100%" fill="url(#svgBg)" />
    <rect width="100%" height="100%" fill="url(#topGlow)" />
    <text x="{width / 2}" y="46" text-anchor="middle"
        font-family="system-ui, 'Segoe UI', Arial, sans-serif"
        font-size="22" font-weight="700" fill="#1e1b4b" letter-
spacing="0.4">{escape(title)}</text>
    <text x="{width / 2}" y="68" text-anchor="middle"
        font-family="system-ui, 'Segoe UI', Arial, sans-serif"
        font-size="12" fill="#6b7280">{escape(x_label)}</text>
    {throughput_panel}
    {collision_panel}
    {delay_panel}
    <text x="{width - 20}" y="{height - 14}" text-anchor="end"
        font-family="system-ui, 'Segoe UI', Arial, sans-serif"
        font-size="10" fill="#9ca3af">csma_ca_sim.py</text>
</svg>
'''

    output_path.parent.mkdir(parents=True, exist_ok=True)
    output_path.write_text(svg, encoding="utf-8")

def build_arg_parser() -> argparse.ArgumentParser:
    """Construit et retourne le parseur d'arguments en ligne de commande.

    Expose tous les paramètres de SimulationConfig ainsi que les options
    de balayage (--sweep-stations, --sweep-wmin) et de répétition (--runs).
    """
    parser = argparse.ArgumentParser(description="Simulateur à événements
discrets du protocole CSMA/CA avec backoff exponentiel binaire")
    # --- Paramètres de la simulation ---

```

```

    parser.add_argument("--stations", type=int, default=8, help="Nombre de
stations en compétition pour le canal")
    parser.add_argument("--arrival-rate", type=float, default=20.0,
help="Taux d'arrivée par station (paquets/s) – processus de renouvellement,
non un Poisson pur")
    parser.add_argument("--simulation-time", type=float, default=20.0,
help="Horizon de simulation (secondes)")
    parser.add_argument("--packet-bits", type=int, default=12000,
help="Taille d'un paquet (bits) – 12 000 bits = 1 500 octets")
    parser.add_argument("--packet-duration", type=float, default=0.001,
help="Durée de transmission d'un paquet (secondes)")
    parser.add_argument("--slot-time", type=float, default=20e-6, help="Durée
d'un slot de backoff (secondes)")
    parser.add_argument("--difs", type=float, default=50e-6, help="DIFS :
délai inter-trames distribué (secondes)")
    parser.add_argument("--sifs", type=float, default=10e-6, help="SIFS :
délai inter-trames court (secondes)")
    # --- Paramètres du protocole CSMA/CA ---
    parser.add_argument("--wmin", type=int, default=15, help="Fenêtre de
contention minimale W_min")
    parser.add_argument("--wmax", type=int, default=1023, help="Fenêtre de
contention maximale W_max")
    parser.add_argument("--kmax", type=int, default=15, help="Nombre maximal
de tentatives avant abandon du paquet")
    # --- Contrôle de la simulation ---
    parser.add_argument("--seed", type=int, default=None, help="Graine
aléatoire pour la reproductibilité (None = non déterministe)")
    parser.add_argument("--runs", type=int, default=1, help="Nombre de
répétitions (résultats moyennés pour réduire la variance)")
    parser.add_argument("--output", type=Path,
default=Path("csma_ca_results.svg"), help="Chemin du fichier SVG généré")
    parser.add_argument("--csv", type=Path, default=None, help="Chemin
optionnel pour sauvegarder les résultats bruts en CSV")
    # --- Mode RTS/CTS ---
    parser.add_argument("--rtscts", action="store_true", help="Active le
mécanisme RTS/CTS avec réservation NAV")
    parser.add_argument("--rts-duration", type=float, default=200e-6,
help="Durée d'une trame RTS (secondes)")
    parser.add_argument("--cts-duration", type=float, default=200e-6,
help="Durée d'une trame CTS (secondes)")

    # --- Balayages paramétriques (mutuellement exclusifs) ---
    sweep_group = parser.add_mutually_exclusive_group()
    sweep_group.add_argument("--sweep-stations", nargs=3, type=int,
metavar=("START", "STOP", "STEP"), help="Balayage du nombre de stations
(début fin pas)")
    sweep_group.add_argument("--sweep-wmin", nargs=3, type=int,
metavar=("START", "STOP", "STEP"), help="Balayage de la fenêtre de contention
minimale W_min (début fin pas)")
    sweep_group.add_argument("--sweep-kmax", nargs=3, type=int,
metavar=("START", "STOP", "STEP"), help="Balayage du nombre maximal de
tentatives K_max (début fin pas)")

    return parser

def main() -> None:

```

```

"""Point d'entrée principal du simulateur.

Parse les arguments, construit la configuration, lance le ou les
balayages
paramétriques ou une simulation simple, puis affiche et sauvegarde les
résultats.
"""
args = build_arg_parser().parse_args()
config = SimulationConfig(
    station_count=args.stations,
    arrival_rate=args.arrival_rate,
    simulation_time=args.simulation_time,
    packet_bits=args.packet_bits,
    packet_duration=args.packet_duration,
    slot_time=args.slot_time,
    difs=args.difs,
    sifs=args.sifs,
    wmin=args.wmin,
    wmax=args.wmax,
    kmax=args.kmax,
    seed=args.seed,
    rtscts=args.rtscts,
    rts_duration=args.rts_duration,
    cts_duration=args.cts_duration,
)

if args.runs <= 0:
    raise SystemExit("--runs must be positive")

if args.sweep_stations is not None:
    start, stop, step = args.sweep_stations
    points = sweep_stations(config, start, stop, step, args.runs)
    print("Station sweep")
    for point in points:
        print(
            f"N={point.x_value:3d} |
throughput={point.throughput_bits_per_s:12.2f} bits/s | "
            f"collision={point.collision_rate * 100:6.2f} % |
delay={point.mean_delay_s * 1000:8.4f} ms"
        )
    plot_points(points, "CSMA/CA : impact du nombre de stations", "Nombre
de stations", args.output)
    print(f"Plot saved to {args.output}")
    if args.csv is not None:
        save_csv(points, None, None, args.csv)
        print(f"CSV saved to {args.csv}")
    return

if args.sweep_wmin is not None:
    start, stop, step = args.sweep_wmin
    points = sweep_wmin(config, start, stop, step, args.runs)
    print("Wmin sweep")
    for point in points:
        print(
            f"Wmin={point.x_value:3d} |
throughput={point.throughput_bits_per_s:12.2f} bits/s | "

```

```

        f"collision={point.collision_rate * 100:6.2f} % |
delay={point.mean_delay_s * 1000:8.4f} ms"
    )
    plot_points(points, "CSMA/CA : impact de la fenêtre de contention
minimale", "Fenêtre de contention minimale (Wmin)", args.output)
    print(f"Plot saved to {args.output}")
    if args.csv is not None:
        save_csv(points, None, None, args.csv)
        print(f"CSV saved to {args.csv}")
    return

if args.sweep_kmax is not None:
    start, stop, step = args.sweep_kmax
    points = sweep_kmax(config, start, stop, step, args.runs)
    print("Kmax sweep")
    for point in points:
        print(
            f"Kmax={point.x_value:3d} |
throughput={point.throughput_bits_per_s:12.2f} bits/s | "
            f"collision={point.collision_rate * 100:6.2f} % |
delay={point.mean_delay_s * 1000:8.4f} ms"
        )
    plot_points(points, "CSMA/CA : impact du nombre maximal de tentatives
(K_max)", "Nombre maximal de tentatives K_max", args.output)
    print(f"Plot saved to {args.output}")
    if args.csv is not None:
        save_csv(points, None, None, args.csv)
        print(f"CSV saved to {args.csv}")
    return

if args.runs == 1:
    result = run_single_experiment(config)
else:
    run_results = [
        run_single_experiment(
            SimulationConfig(
                station_count=config.station_count,
                arrival_rate=config.arrival_rate,
                simulation_time=config.simulation_time,
                packet_bits=config.packet_bits,
                packet_duration=config.packet_duration,
                slot_time=config.slot_time,
                difs=config.difs,
                sifs=config.sifs,
                wmin=config.wmin,
                wmax=config.wmax,
                kmax=config.kmax,
                seed=None if config.seed is None else config.seed +
index,

                rtscts=config.rtscts,
                rts_duration=config.rts_duration,
                cts_duration=config.cts_duration,
            )
        )
        for index in range(args.runs)
    ]
    result = average_results(run_results)

```

```
print_result(config, result)
if args.csv is not None:
    save_csv(None, result, config, args.csv)
    print(f"CSV saved to {args.csv}")

if __name__ == "__main__": # pragma: no cover
    main() # pragma: no cover
```